TCS IPA 35-Mark Java Practice Master Handbook

Complete Edition: single-class problems + class association / aggregation problems

Part 1 - Single-class problems	Page	Part 2 - Aggregation problems	Page
P01 Fruits	2	A01 Vehicle & Insurance Registration	45
P02 Laptop	4	A02 Employee & Project Allocation	48
P03 Player	7	A03 Student & Scholarship Profile	51
P04 Medicine	10	A04 Product & Vendor Logistics	54
P05 Course	12	A05 Hotel Room & Reservation Details	57
P06 Phone	15	A06 Course & Faculty Management	60
P07 Book	17	A07 Property & Owner Records	63
P08 Employee	20	A08 Smart Device & Warranty Plan	66
P09 Movie	23	A09 Order & Shipment Profiles	69
P10 AutonomousCar	26	A10 Book & Publisher Specifications	72
P11 Inventory	29	A11 Movie & Director Affiliation	75
P12 Hotel	32	A12 Flight & Aircraft Fleet	78
P13 NavalVessel	35	A13 Medical Prescription & Doctor Logs	81
P14 Sim	38	A14 Subscription & Account Tier	84
P15 Footwear	41	A15 Inventory Item & Warehouse Location	87
		A16 Bank Account & Branch (single MAX object)	90
		A17 Customer & Address (COUNT result)	93
		A18 Team & Coach Performance	96
		A19 Tablet & Display (MAX by inner attribute)	99
		A20 Library Member & Membership Plan	102
		Final revision checklist	105

Part 1 (pages 1-43) is the original corrected handbook, unchanged. Part 2 (pages 44 onward) adds the 15 aggregation scenarios from your practice set with full statements, sample and edge-case I/O and complete code, plus 5 bonus variations (A16-A20). Page numbers above are the numbers printed in each page header.

TCS IPA 35-Mark Java Practice Master Handbook

Corrected Full Edition - 15 problems, every program complete and tested

What was fixed in this edition

- Problems 2, 3 and 5: the Java listing was cut off in the middle of the POJO class (it stopped after a few getters). The complete class, with every getter and setter, is now included.

- Problem 2: the sample output in the original handbook printed laptops 104, 101, 103, which is NOT descending ID order. The correct output is 104, 103, 101. The program was right; the printed sample was wrong.

- Problems 6 to 15 were listed in the pattern table but never written out. They are now complete, with a problem statement, sample I/O and full code.

- Every program was compiled and run with Java 21 on the sample input AND on an edge case (no match found). The outputs printed in this PDF are the real program outputs, not typed by hand.

- Second-highest logic (Problems 8, 10, 12, 15) now uses the 'first value strictly below the maximum' rule, so duplicate values cannot give a wrong answer.

CRITICAL EXAM HAZARD - Scanner buffer flushing: after sc.nextInt() or sc.nextDouble(), the newline stays in the buffer. Before reading a String with sc.nextLine() you must call sc.nextLine() once to clear it. Every listing here shows exactly where.

Exam template (always in this order)

```
import java.util.*;

public class Solution {
    // 1. static business methods
    // 2. public static void main(String[] args)  -> read input, call methods, print
}
// 3. POJO class: private fields, constructor, getters + setters
```

Reusable patterns

- MAX / MIN with filter: X best = null; if (matches) { if (best == null || v.get() > best.get()) best = v; }

- AVERAGE: return count > 0 ? sum / count : 0; (integer division, guard against zero)

- SORT: list.sort((a, b) -> a.getX() - b.getX()); ascending. Swap a and b for descending. Use Double.compare for doubles.

- SECOND HIGHEST: sort descending, then return the first item whose value < list.get(0)'s value; otherwise null.

- STRINGS: always compare with equalsIgnoreCase(), never ==.

- RETURN null / 0 when nothing matches, and print the exact message the question asks for.

The 15 problems

#	Domain	Method 1	Method 2
1	Fruits	findMaximumPriceByRating	-
2	Laptop	countOfLaptopsByBrand	searchLaptopByOsType
3	Player	findPlayerWithLowestRuns	findAverageRunsByMatchType
4	Medicine	getMedicineByDisease	-
5	Course	findAvgFeeByAdmin	sortCourseByFee
6	Phone	findPriceForGivenBrand	getPhoneByOsAndMinPrice
7	Book	findBookWithMaxPagesByAuthor	countBooksAbovePrice
8	Employee	countEmployeesByBranch	findEmployeeWithSecondHighestSalary
9	Movie	findAvgBudgetByGenre	getMoviesByRatingSortedByBudget
10	AutonomousCar	updateGradeByEnvironment	findCarWithSecondHighestPassRate
11	Inventory	filterItemsNeedingRefill	countByThresholdCategory
12	Hotel	noOfRoomsBookedInGivenMonth	searchHotelWithSecondHighestBill
13	NavalVessel	findAvgVoyagesByVesselClass	findVesselWithHighestVoyagesByMode
14	Sim	transferBalance	changeCircle
15	Footwear	countFootwearByType	getSecondHighestPriceByName

Problems 6 to 15 follow the standard TCS IPA pattern, but exact wording differs between exam papers. Where the table was vague (for example the category rules in Problem 11 or the rating filter in Problem 9) the assumption is stated in the problem. Always follow the wording of YOUR question paper.

Problem 01: Fruits Management System - Problem Statement and I/O

1. Entity attributes (class Fruits)

Class Fruits with private attributes: int fruitId, String fruitName, int price, int rating. Write a parameterized constructor and getters/setters in the same sequence.

2. Static business methods (class Solution)

`findMaximumPriceByRating(Fruits[] arr, int targetRating)`

Returns the maximum-priced Fruits object whose rating is strictly greater than targetRating. If no fruit qualifies, return null.

3. Main execution rules

Read 4 Fruits objects, then targetRating. Call the method. If the result is non-null print fruitId, otherwise print "No such Fruit".

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - nobody has rating above 5

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
555 Apple 200 5 777 Orange 150 4 333 Banana 100 4 888 Avocado 250 4 3	888	555 Apple 200 5 777 Orange 150 4 333 Banana 100 4 888 Avocado 250 4 5	No such Fruit

Explanation: All four fruits have rating > 3. Avocado has the highest price (250), so its ID 888 is printed.

Exam strategy: Start with Fruits maxFruit = null; update when maxFruit == null || f.getPrice() > maxFruit.getPrice().

Problem 01: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the Fruits POJO. Copy as one file, Solution.java.

```
import java.util.Scanner;

public class Solution {

    // 1. STATIC BUSINESS LOGIC FIRST
    public static Fruits findMaximumPriceByRating(Fruits[] arr, int targetRating) {
        Fruits maxFruit = null;
        for (Fruits f : arr) {
            if (f.getRating() > targetRating) {
                if (maxFruit == null || f.getPrice() > maxFruit.getPrice()) {
                    maxFruit = f;
                }
            }
        }
        return maxFruit;
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Fruits[] fruits = new Fruits[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt();
            sc.nextLine();                    // flush newline buffer
            String name = sc.nextLine();
            int price = sc.nextInt();
            int rating = sc.nextInt();
            sc.nextLine();                    // flush newline buffer
            fruits[i] = new Fruits(id, name, price, rating);
        }
        int targetRating = sc.nextInt();

        Fruits res = findMaximumPriceByRating(fruits, targetRating);
        if (res != null) {
            System.out.println(res.getFruitId());
        } else {
            System.out.println("No such Fruit");
        }
        sc.close();
    }
}

// 3. ENTITY POJO CLASS
class Fruits {
    private int fruitId;
    private String fruitName;
    private int price;
    private int rating;

    public Fruits(int fruitId, String fruitName, int price, int rating) {
        this.fruitId = fruitId;
        this.fruitName = fruitName;
        this.price = price;
        this.rating = rating;
    }

    public int getFruitId() { return fruitId; }
    public void setFruitId(int fruitId) { this.fruitId = fruitId; }
    public String getFruitName() { return fruitName; }
    public void setFruitName(String fruitName) { this.fruitName = fruitName; }
    public int getPrice() { return price; }
    public void setPrice(int price) { this.price = price; }
    public int getRating() { return rating; }
    public void setRating(int rating) { this.rating = rating; }
}
```

Problem 02: Laptop Management - Problem Statement and I/O

1. Entity attributes (class Laptop)

Class Laptop with private attributes: int laptopId, String brand, String osType, double price, int rating. Constructor plus getters/setters.

2. Static business methods (class Solution)

`countOfLaptopsByBrand(Laptop[] arr, String brand)`
Returns the count of laptops matching brand (case-insensitive) with rating > 3. If none, return 0.

`searchLaptopByOsType(Laptop[] arr, String osType)`
Returns an array of laptops matching osType (case-insensitive), sorted by laptopId in DESCENDING order. If none, return null.

3. Main execution rules

Read 4 laptops, then searchBrand and searchOs. Print the count (or "The given brand is not available"). Then print each sorted laptop's ID and price on separate lines (or "The given OS is not available").

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - brand and OS not present

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
101 Dell Windows 55000 4 102 Dell Ubuntu 45000 3 103 HP Windows 62000 5 104 Dell Windows 72000 5 Dell Windows	2 104 72000.0 103 62000.0 101 55000.0	101 Dell Windows 55000 4 102 Dell Ubuntu 45000 3 103 HP Windows 62000 5 104 Dell Windows 72000 5 Lenovo Linux	The given brand is not available The given OS is not available

Explanation: Two Dell laptops have rating > 3 (101 and 104), so the count is 2. Windows laptops are 101, 103 and 104; in DESCENDING ID order they print as 104, 103, 101 (each ID followed by its price). NOTE: the original handbook showed 104, 101, 103, which is not descending order - that sample output was wrong.

Exam strategy: Sort descending with list.sort((a, b) -> b.getLaptopId() - a.getLaptopId());

Problem 02: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the Laptop POJO. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHODS FIRST
    public static int countOfLaptopsByBrand(Laptop[] arr, String brand) {
        int count = 0;
        for (Laptop lp : arr) {
            if (lp.getBrand().equalsIgnoreCase(brand) && lp.getRating() > 3) {
                count++;
            }
        }
        return count;
    }

    public static Laptop[] searchLaptopByOsType(Laptop[] arr, String osType) {
        List<Laptop> list = new ArrayList<>();
        for (Laptop lp : arr) {
            if (lp.getOsType().equalsIgnoreCase(osType)) {
                list.add(lp);
            }
        }
        if (list.isEmpty()) return null;
        list.sort((a, b) -> b.getLaptopId() - a.getLaptopId());   // descending
        return list.toArray(new Laptop[0]);
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Laptop[] laptops = new Laptop[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt();
            sc.nextLine();
            String brand = sc.nextLine();
            String os = sc.nextLine();
            double price = sc.nextDouble();
            int rating = sc.nextInt();
            sc.nextLine();
            laptops[i] = new Laptop(id, brand, os, price, rating);
        }
        String searchBrand = sc.nextLine();
        String searchOs = sc.nextLine();

        int count = countOfLaptopsByBrand(laptops, searchBrand);
        if (count > 0) System.out.println(count);
        else System.out.println("The given brand is not available");

        Laptop[] result = searchLaptopByOsType(laptops, searchOs);
        if (result != null) {
            for (Laptop lp : result) {
                System.out.println(lp.getLaptopId());
                System.out.println(lp.getPrice());
            }
        } else {
            System.out.println("The given OS is not available");
        }
        sc.close();
    }
}
// 3. ENTITY POJO CLASS
class Laptop {
    private int laptopId;
    private String brand;
    private String osType;
    private double price;
    private int rating;

    public Laptop(int laptopId, String brand, String osType, double price, int rating) {
        this.laptopId = laptopId;
        this.brand = brand;
        this.osType = osType;
        this.price = price;
        this.rating = rating;
    }

    public int getLaptopId() { return laptopId; }
    public void setLaptopId(int laptopId) { this.laptopId = laptopId; }
    public String getBrand() { return brand; }
```

```
    public void setBrand(String brand) { this.brand = brand; }
    public String getOsType() { return osType; }
    public void setOsType(String osType) { this.osType = osType; }
    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }
    public int getRating() { return rating; }
    public void setRating(int rating) { this.rating = rating; }
}
```

Problem 03: Sports Analytics (Player Runs) - Problem Statement and I/O

1. Entity attributes (class Player)

Class Player with private attributes: int playerId, String playerName, int runs, String playerType, String matchType. Constructor plus getters/setters.

2. Static business methods (class Solution)

`findPlayerWithLowestRuns(Player[] arr, String playerType)`
Returns the Player with the lowest runs whose playerType matches (case-insensitive). If none, return null.

`findAverageRunsByMatchType(Player[] arr, String matchType)`
Returns the INTEGER average runs for the matching matchType. If none, return 0.

3. Main execution rules

Read 4 players, then searchPType and searchMType. Print the lowest player's ID and name on separate lines (or "No such player"). Then print the average runs (or "No such match type").

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - no Allrounder, no Test matches

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
11 Sachin 18000 Batsman ODI 12 Zaheer 1200 Bowler ODI 13 Virat 13000 Batsman ODI 14 Dhoni 10500 Batsman T20 Batsman ODI	14 Dhoni 10733	11 Sachin 18000 Batsman ODI 12 Zaheer 1200 Bowler ODI 13 Virat 13000 Batsman ODI 14 Dhoni 10500 Batsman T20 Allrounder Test	No such player No such match type

Explanation: Lowest-run Batsman is Dhoni (10500). Average for ODI = (18000 + 1200 + 13000) / 3 = 10733 (integer division).

Exam strategy: Division guard: return count > 0 ? (totalRuns / count) : 0; to avoid ArithmeticException.

Problem 03: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the Player POJO. Copy as one file, Solution.java.

```
import java.util.Scanner;

public class Solution {

    // 1. STATIC BUSINESS METHODS FIRST
    public static Player findPlayerWithLowestRuns(Player[] arr, String playerType) {
        Player minPlayer = null;
        for (Player p : arr) {
            if (p.getPlayerType().equalsIgnoreCase(playerType)) {
                if (minPlayer == null || p.getRuns() < minPlayer.getRuns()) {
                    minPlayer = p;
                }
            }
        }
        return minPlayer;
    }

    public static int findAverageRunsByMatchType(Player[] arr, String matchType) {
        int totalRuns = 0;
        int count = 0;
        for (Player p : arr) {
            if (p.getMatchType().equalsIgnoreCase(matchType)) {
                totalRuns += p.getRuns();
                count++;
            }
        }
        return count > 0 ? (totalRuns / count) : 0;
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Player[] players = new Player[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt();
            sc.nextLine();
            String name = sc.nextLine();
            int runs = sc.nextInt();
            sc.nextLine();
            String pType = sc.nextLine();
            String mType = sc.nextLine();
            players[i] = new Player(id, name, runs, pType, mType);
        }
        String searchPType = sc.nextLine();
        String searchMType = sc.nextLine();

        Player lowestPlayer = findPlayerWithLowestRuns(players, searchPType);
        if (lowestPlayer != null) {
            System.out.println(lowestPlayer.getPlayerId());
            System.out.println(lowestPlayer.getPlayerName());
        } else {
            System.out.println("No such player");
        }

        int avgRuns = findAverageRunsByMatchType(players, searchMType);
        if (avgRuns > 0) {
            System.out.println(avgRuns);
        } else {
            System.out.println("No such match type");
        }
        sc.close();
    }
}

// 3. ENTITY POJO CLASS
class Player {
    private int playerId;
    private String playerName;
    private int runs;
    private String playerType;
    private String matchType;

    public Player(int playerId, String playerName, int runs, String playerType, String matchType) {
        this.playerId = playerId;
        this.playerName = playerName;
        this.runs = runs;
        this.playerType = playerType;
        this.matchType = matchType;
    }
```

```
    public int getPlayerId() { return playerId; }
    public void setPlayerId(int playerId) { this.playerId = playerId; }
    public String getPlayerName() { return playerName; }
    public void setPlayerName(String playerName) { this.playerName = playerName; }
    public int getRuns() { return runs; }
    public void setRuns(int runs) { this.runs = runs; }
    public String getPlayerType() { return playerType; }
    public void setPlayerType(String playerType) { this.playerType = playerType; }
    public String getMatchType() { return matchType; }
    public void setMatchType(String matchType) { this.matchType = matchType; }
}
```

Problem 04: Medicine Inventory - Problem Statement and I/O

1. Entity attributes (class Medicine)

Class Medicine with private attributes: String medicineName, String batchNo, String disease, int price. Constructor plus getters/setters.

2. Static business methods (class Solution)

`getMedicineByDisease(Medicine[] arr, String disease)`

Returns an int[] of prices of medicines matching the disease (case-insensitive), sorted ASCENDING. If no medicine matches, return null.

3. Main execution rules

Read 4 medicines, then the target disease. Print each price on a new line. If the result is null print "No Medicine found".

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - disease not present

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
Dolo650 B12 Fever 30 Paracetamol B14 Fever 20 Cetrizine B19 Allergy 45 Azithral B22 Fever 120 Fever	20 30 120	Dolo650 B12 Fever 30 Paracetamol B14 Fever 20 Cetrizine B19 Allergy 45 Azithral B22 Fever 120 Cold	No Medicine found

Explanation: Fever medicines are Paracetamol (20), Dolo650 (30) and Azithral (120), printed in ascending order of price.

Exam strategy: Collect prices into a List<Integer>, call Collections.sort(list), then copy into a new int[].

Problem 04: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the Medicine POJO. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS LOGIC FIRST
    public static int[] getMedicineByDisease(Medicine[] arr, String disease) {
        List<Integer> list = new ArrayList<>();
        for (Medicine m : arr) {
            if (m.getDisease().equalsIgnoreCase(disease)) {
                list.add(m.getPrice());
            }
        }
        if (list.isEmpty()) return null;
        Collections.sort(list);                      // ascending
        int[] result = new int[list.size()];
        for (int i = 0; i < list.size(); i++) {
            result[i] = list.get(i);
        }
        return result;
    }
    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Medicine[] meds = new Medicine[4];
        for (int i = 0; i < 4; i++) {
            String name = sc.nextLine();
            String batch = sc.nextLine();
            String disease = sc.nextLine();
            int price = sc.nextInt();
            sc.nextLine();                           // flush newline buffer
            meds[i] = new Medicine(name, batch, disease, price);
        }
        String targetDisease = sc.nextLine();

        int[] prices = getMedicineByDisease(meds, targetDisease);
        if (prices != null) {
            for (int p : prices) {
                System.out.println(p);
            }
        } else {
            System.out.println("No Medicine found");
        }
        sc.close();
    }
}

// 3. ENTITY POJO CLASS
class Medicine {
    private String medicineName;
    private String batchNo;
    private String disease;
    private int price;

    public Medicine(String medicineName, String batchNo, String disease, int price) {
        this.medicineName = medicineName;
        this.batchNo = batchNo;
        this.disease = disease;
        this.price = price;
    }
    public String getMedicineName() { return medicineName; }
    public void setMedicineName(String medicineName) { this.medicineName = medicineName; }
    public String getBatchNo() { return batchNo; }
    public void setBatchNo(String batchNo) { this.batchNo = batchNo; }
    public String getDisease() { return disease; }
    public void setDisease(String disease) { this.disease = disease; }
    public int getPrice() { return price; }
    public void setPrice(int price) { this.price = price; }
}
```

Problem 05: Course Administration - Problem Statement and I/O

1. Entity attributes (class Course)

Class Course with private attributes: int courseId, String courseName, String courseAdmin, int fee, int rating. Constructor plus getters/setters.

2. Static business methods (class Solution)

`findAvgFeeByAdmin(Course[] arr, String admin)`
Returns the INTEGER average fee of courses by courseAdmin (case-insensitive). If none, return 0.

`sortCourseByFee(Course[] arr, int ratingThreshold)`
Returns an array of courses with rating <= ratingThreshold, sorted by fee ASCENDING. If none, return null.

3. Main execution rules

Read 4 courses, then admin name and rating threshold. Print the average fee (or "No Such Courses"). Then print matching course names one per line (or "No certificate found").

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - unknown admin, rating threshold 1

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
101 Java John 15000 4 102 Python John 12000 5 103 SQL Alex 8000 3 104 AWS John 20000 3 John 4	15666 SQL Java AWS	101 Java John 15000 4 102 Python John 12000 5 103 SQL Alex 8000 3 104 AWS John 20000 3 Zoe 1	No Such Courses No certificate found

Explanation: John's average fee = (15000 + 12000 + 20000) / 3 = 15666. Courses with rating <= 4 sorted by fee: SQL (8000), Java (15000), AWS (20000).

Exam strategy: Ascending sort: list.sort((a, b) -> a.getFee() - b.getFee());

Problem 05: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the Course POJO. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHODS FIRST
    public static int findAvgFeeByAdmin(Course[] arr, String admin) {
        int sum = 0, count = 0;
        for (Course c : arr) {
            if (c.getCourseAdmin().equalsIgnoreCase(admin)) {
                sum += c.getFee();
                count++;
            }
        }
        return count > 0 ? (sum / count) : 0;
    }

    public static Course[] sortCourseByFee(Course[] arr, int ratingThreshold) {
        List<Course> list = new ArrayList<>();
        for (Course c : arr) {
            if (c.getRating() <= ratingThreshold) {
                list.add(c);
            }
        }
        if (list.isEmpty()) return null;
        list.sort((a, b) -> a.getFee() - b.getFee());             // ascending by fee
        return list.toArray(new Course[0]);
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Course[] courses = new Course[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt();
            sc.nextLine();
            String name = sc.nextLine();
            String admin = sc.nextLine();
            int fee = sc.nextInt();
            int rating = sc.nextInt();
            sc.nextLine();
            courses[i] = new Course(id, name, admin, fee, rating);
        }
        String targetAdmin = sc.nextLine();
        int targetRating = sc.nextInt();

        int avg = findAvgFeeByAdmin(courses, targetAdmin);
        if (avg > 0) System.out.println(avg);
        else System.out.println("No Such Courses");

        Course[] res = sortCourseByFee(courses, targetRating);
        if (res != null) {
            for (Course c : res) System.out.println(c.getCourseName());
        } else {
            System.out.println("No certificate found");
        }
        sc.close();
    }
}

// 3. ENTITY POJO CLASS
class Course {
    private int courseId;
    private String courseName;
    private String courseAdmin;
    private int fee;
    private int rating;

    public Course(int courseId, String courseName, String courseAdmin, int fee, int rating) {
        this.courseId = courseId;
        this.courseName = courseName;
        this.courseAdmin = courseAdmin;
        this.fee = fee;
        this.rating = rating;
    }

    public int getCourseId() { return courseId; }
    public void setCourseId(int courseId) { this.courseId = courseId; }
    public String getCourseName() { return courseName; }
    public void setCourseName(String courseName) { this.courseName = courseName; }
    public String getCourseAdmin() { return courseAdmin; }
```

```
    public void setCourseAdmin(String courseAdmin) { this.courseAdmin = courseAdmin; }
    public int getFee() { return fee; }
    public void setFee(int fee) { this.fee = fee; }
    public int getRating() { return rating; }
    public void setRating(int rating) { this.rating = rating; }
}
```

Problem 06: Phone Store - Problem Statement and I/O

1. Entity attributes (class Phone)

Class Phone with private attributes: int phoneId, String os, String brand, int price. Constructor plus getters/setters.

2. Static business methods (class Solution)

`findPriceForGivenBrand(Phone[] arr, String brand)`
Returns the TOTAL price of all phones of the given brand (case-insensitive). If none, return 0.

`getPhoneByOsAndMinPrice(Phone[] arr, String os)`
Returns the phone with the given OS (case-insensitive) that has the MINIMUM price. If none, return null.

3. Main execution rules

Read 4 phones (id, os, brand, price), then brand and os. Print the total price (or "The given Brand is not available"). Then print the matching phone's ID (or "The given OS is not available").

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - brand and OS not present

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
1 Android Samsung 20000 2 iOS Apple 70000 3 Android Samsung 15000 4 Android OnePlus 30000 Samsung Android	35000 3	1 Android Samsung 20000 2 iOS Apple 70000 3 Android Samsung 15000 4 Android OnePlus 30000 Nokia Windows	The given Brand is not availab le The given OS is not available

Explanation: Samsung total = 20000 + 15000 = 35000. Among Android phones the cheapest is phone 3 (15000).

Exam strategy: Min-tracking pattern: Phone min = null; if (min == null || p.getPrice() < min.getPrice()) min = p;

Problem 06: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the Phone POJO. Copy as one file, Solution.java.

```
import java.util.Scanner;

public class Solution {

    // 1. STATIC BUSINESS METHODS FIRST
    public static int findPriceForGivenBrand(Phone[] arr, String brand) {
        int total = 0;
        for (Phone p : arr) {
            if (p.getBrand().equalsIgnoreCase(brand)) {
                total += p.getPrice();
            }
        }
        return total;
    }

    public static Phone getPhoneByOsAndMinPrice(Phone[] arr, String os) {
        Phone min = null;
        for (Phone p : arr) {
            if (p.getOs().equalsIgnoreCase(os)) {
                if (min == null || p.getPrice() < min.getPrice()) {
                    min = p;
                }
            }
        }
        return min;
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Phone[] phones = new Phone[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt();
            sc.nextLine();
            String os = sc.nextLine();
            String brand = sc.nextLine();
            int price = sc.nextInt();
            sc.nextLine();
            phones[i] = new Phone(id, os, brand, price);
        }
        String searchBrand = sc.nextLine();
        String searchOs = sc.nextLine();

        int total = findPriceForGivenBrand(phones, searchBrand);
        if (total > 0) System.out.println(total);
        else System.out.println("The given Brand is not available");

        Phone res = getPhoneByOsAndMinPrice(phones, searchOs);
        if (res != null) System.out.println(res.getPhoneId());
        else System.out.println("The given OS is not available");
        sc.close();
    }
}

// 3. ENTITY POJO CLASS
class Phone {
    private int phoneId;
    private String os;
    private String brand;
    private int price;
    public Phone(int phoneId, String os, String brand, int price) {
        this.phoneId = phoneId;
        this.os = os;
        this.brand = brand;
        this.price = price;
    }

    public int getPhoneId() { return phoneId; }
    public void setPhoneId(int phoneId) { this.phoneId = phoneId; }
    public String getOs() { return os; }
    public void setOs(String os) { this.os = os; }
    public String getBrand() { return brand; }
    public void setBrand(String brand) { this.brand = brand; }
    public int getPrice() { return price; }
    public void setPrice(int price) { this.price = price; }
}
```

Problem 07: Book Library - Problem Statement and I/O

1. Entity attributes (class Book)

Class Book with private attributes: int bookId, String title, String author, double price, int pages. Constructor plus getters/setters.

2. Static business methods (class Solution)

`findBookWithMaxPagesByAuthor(Book[] arr, String author)`
Returns the Book with the MAXIMUM pages written by the given author (case-insensitive). If none, return null.

`countBooksAbovePrice(Book[] arr, double threshold)`
Returns the count of books whose price is strictly greater than threshold. If none, return 0.

3. Main execution rules

Read 4 books, then author and price threshold. Print the book's ID and title on separate lines (or "No such book"). Then print the count (or "No books found").

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - unknown author, very high threshold

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
1 JavaBasics Gosling 450.5 300 2 DeepJava Gosling 700 520 3 Python Guido 500 410 4 Algorithms Cormen 900 1200 Gosling 480	2 DeepJava 3	1 JavaBasics Gosling 450.5 300 2 DeepJava Gosling 700 520 3 Python Guido 500 410 4 Algorithms Cormen 900 1200 Knuth 5000	No such book No books found

Explanation: Gosling's book with most pages is DeepJava (520 pages, ID 2). Books priced above 480: DeepJava (700), Python (500), Algorithms (900) = 3.

Exam strategy: Read price with sc.nextDouble() and pages with sc.nextInt(), then flush with sc.nextLine().

Problem 07: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the Book POJO. Copy as one file, Solution.java.

```
import java.util.Scanner;

public class Solution {

    // 1. STATIC BUSINESS METHODS FIRST
    public static Book findBookWithMaxPagesByAuthor(Book[] arr, String author) {
        Book maxBook = null;
        for (Book b : arr) {
            if (b.getAuthor().equalsIgnoreCase(author)) {
                if (maxBook == null || b.getPages() > maxBook.getPages()) {
                    maxBook = b;
                }
            }
        }
        return maxBook;
    }

    public static int countBooksAbovePrice(Book[] arr, double threshold) {
        int count = 0;
        for (Book b : arr) {
            if (b.getPrice() > threshold) {
                count++;
            }
        }
        return count;
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Book[] books = new Book[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt();
            sc.nextLine();
            String title = sc.nextLine();
            String author = sc.nextLine();
            double price = sc.nextDouble();
            int pages = sc.nextInt();
            sc.nextLine();
            books[i] = new Book(id, title, author, price, pages);
        }
        String searchAuthor = sc.nextLine();
        double threshold = sc.nextDouble();

        Book res = findBookWithMaxPagesByAuthor(books, searchAuthor);
        if (res != null) {
            System.out.println(res.getBookId());
            System.out.println(res.getTitle());
        } else {
            System.out.println("No such book");
        }

        int count = countBooksAbovePrice(books, threshold);
        if (count > 0) System.out.println(count);
        else System.out.println("No books found");
        sc.close();
    }
}

// 3. ENTITY POJO CLASS
class Book {
    private int bookId;
    private String title;
    private String author;
    private double price;
    private int pages;

    public Book(int bookId, String title, String author, double price, int pages) {
        this.bookId = bookId;
        this.title = title;
        this.author = author;
        this.price = price;
        this.pages = pages;
    }

    public int getBookId() { return bookId; }
    public void setBookId(int bookId) { this.bookId = bookId; }
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }
    public String getAuthor() { return author; }
```

```
    public void setAuthor(String author) { this.author = author; }
    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }
    public int getPages() { return pages; }
    public void setPages(int pages) { this.pages = pages; }
}
```

Problem 08: Employee Performance - Problem Statement and I/O

1. Entity attributes (class Employee)

Class Employee with private attributes: int empId, String empName, String branch, int rating, double salary. Constructor plus getters/setters.

2. Static business methods (class Solution)

`countEmployeesByBranch(Employee[] arr, String branch)`
Returns the count of employees in the given branch (case-insensitive) with rating >= 4. If none, return 0.

`findEmployeeWithSecondHighestSalary(Employee[] arr)`
Returns the employee with the SECOND HIGHEST distinct salary. If there is no second distinct salary, return null.

3. Main execution rules

Read 4 employees, then branch. Print the count (or "No employee found"). Then print the second-highest-salary employee's ID and name on separate lines (or "No such employee").

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - all salaries equal, branch not present

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
201 Ravi IT 5 60000 202 Meena HR 4 45000 203 Kiran IT 3 80000 204 Anil IT 4 70000 IT	2 204 Anil	301 A IT 5 50000 302 B IT 5 50000 303 C HR 4 50000 304 D HR 4 50000 Sales	No employee found No such employee

Explanation: IT employees with rating >= 4: Ravi (5) and Anil (4) = 2. Salaries sorted: 80000 (Kiran), 70000 (Anil) -> second highest is Anil, ID 204.

Exam strategy: Sort a copy DESCENDING, then return the first employee whose salary is strictly less than the top salary. This handles duplicate salaries.

Problem 08: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the Employee POJO. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHODS FIRST
    public static int countEmployeesByBranch(Employee[] arr, String branch) {
        int count = 0;
        for (Employee e : arr) {
            if (e.getBranch().equalsIgnoreCase(branch) && e.getRating() >= 4) {
                count++;
            }
        }
        return count;
    }

    public static Employee findEmployeeWithSecondHighestSalary(Employee[] arr) {
        List<Employee> list = new ArrayList<>(Arrays.asList(arr));
        if (list.isEmpty()) return null;
        list.sort((a, b) -> Double.compare(b.getSalary(), a.getSalary()));  // descending
        double top = list.get(0).getSalary();
        for (Employee e : list) {
            if (e.getSalary() < top) {
                return e;                       // first salary below the maximum
            }
        }
        return null;                            // all salaries are equal
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Employee[] emps = new Employee[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt();
            sc.nextLine();
            String name = sc.nextLine();
            String branch = sc.nextLine();
            int rating = sc.nextInt();
            double salary = sc.nextDouble();
            sc.nextLine();
            emps[i] = new Employee(id, name, branch, rating, salary);
        }
        String searchBranch = sc.nextLine();

        int count = countEmployeesByBranch(emps, searchBranch);
        if (count > 0) System.out.println(count);
        else System.out.println("No employee found");

        Employee second = findEmployeeWithSecondHighestSalary(emps);
        if (second != null) {
            System.out.println(second.getEmpId());
            System.out.println(second.getEmpName());
        } else {
            System.out.println("No such employee");
        }
        sc.close();
    }
}

// 3. ENTITY POJO CLASS
class Employee {
    private int empId;
    private String empName;
    private String branch;
    private int rating;
    private double salary;

    public Employee(int empId, String empName, String branch, int rating, double salary) {
        this.empId = empId;
        this.empName = empName;
        this.branch = branch;
        this.rating = rating;
        this.salary = salary;
    }

    public int getEmpId() { return empId; }
    public void setEmpId(int empId) { this.empId = empId; }
    public String getEmpName() { return empName; }
    public void setEmpName(String empName) { this.empName = empName; }
    public String getBranch() { return branch; }
```

```
    public void setBranch(String branch) { this.branch = branch; }
    public int getRating() { return rating; }
    public void setRating(int rating) { this.rating = rating; }
    public double getSalary() { return salary; }
    public void setSalary(double salary) { this.salary = salary; }
}
```

Problem 09: Movie Budget Analysis - Problem Statement and I/O

1. Entity attributes (class Movie)

Class Movie with private attributes: String movieName, String company, String genre, int budget, int rating. Constructor plus getters/setters.

2. Static business methods (class Solution)

`findAvgBudgetByGenre(Movie[] arr, String genre)`
Returns the INTEGER average budget of movies of the given genre (case-insensitive). If none, return 0.

`getMoviesByRatingSortedByBudget(Movie[] arr, int rating)`
Returns movies with rating >= the given rating, sorted by budget in DESCENDING order. If none, return null.

3. Main execution rules

Read 4 movies, then genre and rating. Print the average budget (or "No such genre"). Then print the matching movie names one per line (or "No movies found").

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - unknown genre, rating 6

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
Inception WB SciFi 160 5 Avatar Fox SciFi 237 4 Titanic Paramount Romance 200 4 Dunkirk WB War 100 3 SciFi 4	198 Avatar Titanic Inception	Inception WB SciFi 160 5 Avatar Fox SciFi 237 4 Titanic Paramount Romance 200 4 Dunkirk WB War 100 3 Horror 6	No such genre No movies found

Explanation: SciFi average = (160 + 237) / 2 = 198 (integer division). Movies with rating >= 4 by budget descending: Avatar (237), Titanic (200), Inception (160).

Exam strategy: If your exam says 'rating equals', change >= to == in the filter; everything else stays the same.

Problem 09: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the Movie POJO. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHODS FIRST
    public static int findAvgBudgetByGenre(Movie[] arr, String genre) {
        int sum = 0, count = 0;
        for (Movie m : arr) {
            if (m.getGenre().equalsIgnoreCase(genre)) {
                sum += m.getBudget();
                count++;
            }
        }
        return count > 0 ? (sum / count) : 0;
    }

    public static Movie[] getMoviesByRatingSortedByBudget(Movie[] arr, int rating) {
        List<Movie> list = new ArrayList<>();
        for (Movie m : arr) {
            if (m.getRating() >= rating) {
                list.add(m);
            }
        }
        if (list.isEmpty()) return null;
        list.sort((a, b) -> b.getBudget() - a.getBudget());       // descending
        return list.toArray(new Movie[0]);
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Movie[] movies = new Movie[4];
        for (int i = 0; i < 4; i++) {
            String name = sc.nextLine();
            String company = sc.nextLine();
            String genre = sc.nextLine();
            int budget = sc.nextInt();
            int rating = sc.nextInt();
            sc.nextLine();
            movies[i] = new Movie(name, company, genre, budget, rating);
        }
        String searchGenre = sc.nextLine();
        int searchRating = sc.nextInt();

        int avg = findAvgBudgetByGenre(movies, searchGenre);
        if (avg > 0) System.out.println(avg);
        else System.out.println("No such genre");

        Movie[] res = getMoviesByRatingSortedByBudget(movies, searchRating);
        if (res != null) {
            for (Movie m : res) System.out.println(m.getMovieName());
        } else {
            System.out.println("No movies found");
        }
        sc.close();
    }
}

// 3. ENTITY POJO CLASS
class Movie {
    private String movieName;
    private String company;
    private String genre;
    private int budget;
    private int rating;

    public Movie(String movieName, String company, String genre, int budget, int rating) {
        this.movieName = movieName;
        this.company = company;
        this.genre = genre;
        this.budget = budget;
        this.rating = rating;
    }

    public String getMovieName() { return movieName; }
    public void setMovieName(String movieName) { this.movieName = movieName; }
    public String getCompany() { return company; }
    public void setCompany(String company) { this.company = company; }
    public String getGenre() { return genre; }
    public void setGenre(String genre) { this.genre = genre; }
```

```
    public int getBudget() { return budget; }
    public void setBudget(int budget) { this.budget = budget; }
    public int getRating() { return rating; }
    public void setRating(int rating) { this.rating = rating; }
}
```

Problem 10: Autonomous Car Testing - Problem Statement and I/O

1. Entity attributes (class AutonomousCar)

Class AutonomousCar with private attributes: int carId, String brand, int testsConducted, int testsPassed, String environment, String grade. The constructor takes only the first FIVE values; grade is initialised to "NA" inside the constructor. Getters/setters for all six.

2. Static business methods (class Solution)

`updateGradeByEnvironment(AutonomousCar[] arr, String env)`

For every car in the given environment (case-insensitive) compute passRate = testsPassed * 100 / testsConducted. Set grade = "A1" if passRate >= 80, else "B2". Return the updated cars; if none, return null.

`findCarWithSecondHighestPassRate(AutonomousCar[] arr)`

Returns the car with the SECOND HIGHEST distinct pass rate (all cars). If there is no second distinct rate, return null.

3. Main execution rules

Read 4 cars (id, brand, tests, passed, environment), then env. Print each updated car's ID and grade on separate lines (or "No car found in the given environment"). Then print the second-highest car's ID and brand (or "No such car").

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - unknown environment, all pass rates equal

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
1 Tesla 100 90 Urban 2 Waymo 200 150 Urban 3 Cruise 50 48 Highway 4 Zoox 80 40 Urban Urban	1 A1 2 B2 4 B2 1 Tesla	1 A 10 5 Urban 2 B 20 10 Urban 3 C 30 15 Urban 4 D 40 20 Urban Rural	No car found in the given environment No such car

Explanation: Urban cars: Tesla 90% -> A1, Waymo 75% -> B2, Zoox 50% -> B2. All pass rates: 90, 75, 96, 50 -> second highest is Tesla (90%), ID 1.

Exam strategy: Use 100.0 (a double) when computing the rate so integer division does not truncate it; guard testsConducted == 0.

Problem 10: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the AutonomousCar POJO. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // helper: pass percentage of one car (guards against divide by zero)
    private static double passRate(AutonomousCar c) {
        if (c.getTestsConducted() == 0) return 0;
        return c.getTestsPassed() * 100.0 / c.getTestsConducted();
    }
    // 1. STATIC BUSINESS METHODS FIRST
    public static AutonomousCar[] updateGradeByEnvironment(AutonomousCar[] arr, String env) {
        List<AutonomousCar> list = new ArrayList<>();
        for (AutonomousCar c : arr) {
            if (c.getEnvironment().equalsIgnoreCase(env)) {
                c.setGrade(passRate(c) >= 80 ? "A1" : "B2");
                list.add(c);
            }
        }
        if (list.isEmpty()) return null;
        return list.toArray(new AutonomousCar[0]);
    }

    public static AutonomousCar findCarWithSecondHighestPassRate(AutonomousCar[] arr) {
        List<AutonomousCar> list = new ArrayList<>(Arrays.asList(arr));
        if (list.isEmpty()) return null;
        list.sort((a, b) -> Double.compare(passRate(b), passRate(a)));   // descending
        double top = passRate(list.get(0));
        for (AutonomousCar c : list) {
            if (passRate(c) < top) {
                return c;
            }
        }
        return null;
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        AutonomousCar[] cars = new AutonomousCar[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt();
            sc.nextLine();
            String brand = sc.nextLine();
            int tests = sc.nextInt();
            int passed = sc.nextInt();
            sc.nextLine();
            String env = sc.nextLine();
            cars[i] = new AutonomousCar(id, brand, tests, passed, env);
        }
        String searchEnv = sc.nextLine();

        AutonomousCar[] updated = updateGradeByEnvironment(cars, searchEnv);
        if (updated != null) {
            for (AutonomousCar c : updated) {
                System.out.println(c.getCarId());
                System.out.println(c.getGrade());
            }
        } else {
            System.out.println("No car found in the given environment");
        }

        AutonomousCar second = findCarWithSecondHighestPassRate(cars);
        if (second != null) {
            System.out.println(second.getCarId());
            System.out.println(second.getBrand());
        } else {
            System.out.println("No such car");
        }
        sc.close();
    }
}

// 3. ENTITY POJO CLASS
class AutonomousCar {
    private int carId;
    private String brand;
    private int testsConducted;
    private int testsPassed;
    private String environment;
```

```
    private String grade;

    public AutonomousCar(int carId, String brand, int testsConducted,
            int testsPassed, String environment) {
        this.carId = carId;
        this.brand = brand;
        this.testsConducted = testsConducted;
        this.testsPassed = testsPassed;
        this.environment = environment;
        this.grade = "NA";
    }

    public int getCarId() { return carId; }
    public void setCarId(int carId) { this.carId = carId; }
    public String getBrand() { return brand; }
    public void setBrand(String brand) { this.brand = brand; }
    public int getTestsConducted() { return testsConducted; }
    public void setTestsConducted(int testsConducted) { this.testsConducted = testsConducted; }
    public int getTestsPassed() { return testsPassed; }
    public void setTestsPassed(int testsPassed) { this.testsPassed = testsPassed; }
    public String getEnvironment() { return environment; }
    public void setEnvironment(String environment) { this.environment = environment; }
    public String getGrade() { return grade; }
    public void setGrade(String grade) { this.grade = grade; }
}
```

Problem 11: Inventory Replenishment - Problem Statement and I/O

1. Entity attributes (class Inventory)

Class Inventory with private attributes: String inventoryId, int maxQty, int curQty, int threshold. Constructor plus getters/setters.

2. Static business methods (class Solution)

`filterItemsNeedingRefill(Inventory[] arr)`

Returns the items whose curQty <= threshold (they need a refill), in input order. If none, return null.

`countByThresholdCategory(Inventory[] arr, String category)`

Category by threshold value: "Critical" if threshold > 75, "Moderate" if 50 to 75 (inclusive), "Normal" if below 50. Returns how many items fall in the given category (case-insensitive). If none, return 0.

3. Main execution rules

Read 4 items (id, max, current, threshold), then category. Print the IDs needing refill one per line (or "No items need refill"). Then print the category count (or "No items in the given category").

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - no refills needed, empty category

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
I101 100 20 30 I102 200 150 80 I103 150 60 60 I104 300 90 90 Critical	I101 I103 I104 2	I201 100 90 30 I202 100 80 20 I203 100 70 10 I204 100 60 40 Critical	No items need refill No items in the given category

Explanation: Refill needed (cur <= threshold): I101 (20<=30), I103 (60<=60), I104 (90<=90). Critical (threshold > 75): I102 (80) and I104 (90) = 2.

Exam strategy: Note the <= : an item exactly AT its threshold also needs a refill (I103 and I104 in the sample).

Problem 11: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the Inventory POJO. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // helper: classify a threshold value
    private static String categoryOf(int threshold) {
        if (threshold > 75) return "Critical";
        if (threshold >= 50) return "Moderate";
        return "Normal";
    }
    // 1. STATIC BUSINESS METHODS FIRST
    public static Inventory[] filterItemsNeedingRefill(Inventory[] arr) {
        List<Inventory> list = new ArrayList<>();
        for (Inventory inv : arr) {
            if (inv.getCurQty() <= inv.getThreshold()) {
                list.add(inv);
            }
        }
        if (list.isEmpty()) return null;
        return list.toArray(new Inventory[0]);
    }

    public static int countByThresholdCategory(Inventory[] arr, String category) {
        int count = 0;
        for (Inventory inv : arr) {
            if (categoryOf(inv.getThreshold()).equalsIgnoreCase(category)) {
                count++;
            }
        }
        return count;
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Inventory[] items = new Inventory[4];
        for (int i = 0; i < 4; i++) {
            String id = sc.nextLine();
            int max = sc.nextInt();
            int cur = sc.nextInt();
            int thr = sc.nextInt();
            sc.nextLine();
            items[i] = new Inventory(id, max, cur, thr);
        }
        String category = sc.nextLine();

        Inventory[] refill = filterItemsNeedingRefill(items);
        if (refill != null) {
            for (Inventory inv : refill) System.out.println(inv.getInventoryId());
        } else {
            System.out.println("No items need refill");
        }

        int count = countByThresholdCategory(items, category);
        if (count > 0) System.out.println(count);
        else System.out.println("No items in the given category");
        sc.close();
    }
}
// 3. ENTITY POJO CLASS
class Inventory {
    private String inventoryId;
    private int maxQty;
    private int curQty;
    private int threshold;

    public Inventory(String inventoryId, int maxQty, int curQty, int threshold) {
        this.inventoryId = inventoryId;
        this.maxQty = maxQty;
        this.curQty = curQty;
        this.threshold = threshold;
    }

    public String getInventoryId() { return inventoryId; }
    public void setInventoryId(String inventoryId) { this.inventoryId = inventoryId; }
    public int getMaxQty() { return maxQty; }
    public void setMaxQty(int maxQty) { this.maxQty = maxQty; }
    public int getCurQty() { return curQty; }
```

```
    public void setCurQty(int curQty) { this.curQty = curQty; }
    public int getThreshold() { return threshold; }
    public void setThreshold(int threshold) { this.threshold = threshold; }
}
```

Problem 12: Hotel Booking - Problem Statement and I/O

1. Entity attributes (class Hotel)

Class Hotel with private attributes: int hotelId, String hotelName, String bookingDate (format dd-MM-yyyy), int roomsBooked, String wifi, double totalBill. Constructor plus getters/setters.

2. Static business methods (class Solution)

`noOfRoomsBookedInGivenMonth(Hotel[] arr, String month)`
Returns the TOTAL rooms booked in the given month. The month may be a full name (March) or 3-letter short form (Mar), case-insensitive. If none, return 0.

`searchHotelWithSecondHighestBill(Hotel[] arr)`
Returns the hotel with the SECOND HIGHEST distinct totalBill. If there is no second distinct bill, return null.

3. Main execution rules

Read 4 hotels, then month. Print the total rooms (or "No rooms booked in the given month"). Then print the second-highest-bill hotel's ID and name on separate lines (or "No such hotel").

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - no bookings in December, all bills equal

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
1 Taj 05-03-2023 3 yes 12000 2 Oberoi 15-03-2023 2 no 18000.5 3 Hyatt 20-04-2023 4 yes 15000 4 ITC 25-03-2023 1 yes 9000 March	6 3 Hyatt	1 Taj 05-03-2023 3 yes 10000 2 Oberoi 15-03-2023 2 no 10000 3 Hyatt 20-04-2023 4 yes 10000 4 ITC 25-03-2023 1 yes 10000 December	No rooms booked in the given month No such hotel

Explanation: March bookings: Taj (3) + Oberoi (2) + ITC (1) = 6 rooms. Bills sorted: 18000.5 (Oberoi), 15000 (Hyatt) -> second highest is Hyatt, ID 3.

Exam strategy: Parse the date with LocalDate.parse(date, DateTimeFormatter.ofPattern("dd-MM-yyyy")) and compare getMonth().name().

Problem 12: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the Hotel POJO. Copy as one file, Solution.java.

```
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.util.*;

public class Solution {

    // helper: does the booking date fall in the given month (Mar or March)?
    private static boolean isInMonth(String date, String month) {
        try {
            LocalDate d = LocalDate.parse(date, DateTimeFormatter.ofPattern("dd-MM-yyyy"));
            String full = d.getMonth().name();                      // e.g. MARCH
            return full.equalsIgnoreCase(month)
                || full.substring(0, 3).equalsIgnoreCase(month);    // MAR
        } catch (DateTimeParseException e) {
            return false;
        }
    }

    // 1. STATIC BUSINESS METHODS FIRST
    public static int noOfRoomsBookedInGivenMonth(Hotel[] arr, String month) {
        int total = 0;
        for (Hotel h : arr) {
            if (isInMonth(h.getBookingDate(), month)) {
                total += h.getRoomsBooked();
            }
        }
        return total;
    }
    public static Hotel searchHotelWithSecondHighestBill(Hotel[] arr) {
        List<Hotel> list = new ArrayList<>(Arrays.asList(arr));
        if (list.isEmpty()) return null;
        list.sort((a, b) -> Double.compare(b.getTotalBill(), a.getTotalBill()));  // descending
        double top = list.get(0).getTotalBill();
        for (Hotel h : list) {
            if (h.getTotalBill() < top) {
                return h;
            }
        }
        return null;
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Hotel[] hotels = new Hotel[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt();
            sc.nextLine();
            String name = sc.nextLine();
            String date = sc.nextLine();
            int rooms = sc.nextInt();
            sc.nextLine();
            String wifi = sc.nextLine();
            double bill = sc.nextDouble();
            sc.nextLine();
            hotels[i] = new Hotel(id, name, date, rooms, wifi, bill);
        }
        String month = sc.nextLine();
        int rooms = noOfRoomsBookedInGivenMonth(hotels, month);
        if (rooms > 0) System.out.println(rooms);
        else System.out.println("No rooms booked in the given month");

        Hotel second = searchHotelWithSecondHighestBill(hotels);
        if (second != null) {
            System.out.println(second.getHotelId());
            System.out.println(second.getHotelName());
        } else {
            System.out.println("No such hotel");
        }
        sc.close();
    }
}

// 3. ENTITY POJO CLASS
class Hotel {
    private int hotelId;
    private String hotelName;
```

```
    private String bookingDate;
    private int roomsBooked;
    private String wifi;
    private double totalBill;

    public Hotel(int hotelId, String hotelName, String bookingDate, int roomsBooked,
            String wifi, double totalBill) {
        this.hotelId = hotelId;
        this.hotelName = hotelName;
        this.bookingDate = bookingDate;
        this.roomsBooked = roomsBooked;
        this.wifi = wifi;
        this.totalBill = totalBill;
    }

    public int getHotelId() { return hotelId; }
    public void setHotelId(int hotelId) { this.hotelId = hotelId; }
    public String getHotelName() { return hotelName; }
    public void setHotelName(String hotelName) { this.hotelName = hotelName; }
    public String getBookingDate() { return bookingDate; }
    public void setBookingDate(String bookingDate) { this.bookingDate = bookingDate; }
    public int getRoomsBooked() { return roomsBooked; }
    public void setRoomsBooked(int roomsBooked) { this.roomsBooked = roomsBooked; }
    public String getWifi() { return wifi; }
    public void setWifi(String wifi) { this.wifi = wifi; }
    public double getTotalBill() { return totalBill; }
    public void setTotalBill(double totalBill) { this.totalBill = totalBill; }
}
```

Problem 13: Naval Vessel Fleet - Problem Statement and I/O

1. Entity attributes (class NavalVessel)

Class NavalVessel with private attributes: int vesselId, String vesselName, int voyages, String mode, String vesselClass. Constructor plus getters/setters.

2. Static business methods (class Solution)

`findAvgVoyagesByVesselClass(NavalVessel[] arr, String vesselClass)`
Returns the INTEGER average voyages of vessels of the given class (case-insensitive). If none, return 0.

`findVesselWithHighestVoyagesByMode(NavalVessel[] arr, String mode)`
Returns the vessel with the HIGHEST voyages among vessels with the given mode (case-insensitive). If none, return null.

3. Main execution rules

Read 4 vessels, then vessel class and mode. Print the average voyages (or "No such vessel class"). Then print the vessel's ID and name on separate lines (or "No such mode").

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - unknown class and mode

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
1 Vikrant 40 Combat Carrier 2 Kolkata 30 Combat Destroyer 3 Sandhayak 25 Survey Support 4 Vikramaditya 50 Combat Carrier Carrier Combat	45 4 Vikramaditya	1 Vikrant 40 Combat Carrier 2 Kolkata 30 Combat Destroyer 3 Sandhayak 25 Survey Support 4 Vikramaditya 50 Combat Carrier Submarine Rescue	No such vessel class No such mode

Explanation: Carrier average = (40 + 50) / 2 = 45. In Combat mode the highest voyages is Vikramaditya (50), ID 4.

Exam strategy: Same 'max with filter' skeleton as Problems 1 and 7: null check first, then compare.

Problem 13: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the NavalVessel POJO. Copy as one file, Solution.java.

```
import java.util.Scanner;

public class Solution {

    // 1. STATIC BUSINESS METHODS FIRST
    public static int findAvgVoyagesByVesselClass(NavalVessel[] arr, String vesselClass) {
        int sum = 0, count = 0;
        for (NavalVessel v : arr) {
            if (v.getVesselClass().equalsIgnoreCase(vesselClass)) {
                sum += v.getVoyages();
                count++;
            }
        }
        return count > 0 ? (sum / count) : 0;
    }

    public static NavalVessel findVesselWithHighestVoyagesByMode(NavalVessel[] arr, String mode) {
        NavalVessel max = null;
        for (NavalVessel v : arr) {
            if (v.getMode().equalsIgnoreCase(mode)) {
                if (max == null || v.getVoyages() > max.getVoyages()) {
                    max = v;
                }
            }
        }
        return max;
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        NavalVessel[] vessels = new NavalVessel[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt();
            sc.nextLine();
            String name = sc.nextLine();
            int voyages = sc.nextInt();
            sc.nextLine();
            String mode = sc.nextLine();
            String cls = sc.nextLine();
            vessels[i] = new NavalVessel(id, name, voyages, mode, cls);
        }
        String searchClass = sc.nextLine();
        String searchMode = sc.nextLine();

        int avg = findAvgVoyagesByVesselClass(vessels, searchClass);
        if (avg > 0) System.out.println(avg);
        else System.out.println("No such vessel class");

        NavalVessel res = findVesselWithHighestVoyagesByMode(vessels, searchMode);
        if (res != null) {
            System.out.println(res.getVesselId());
            System.out.println(res.getVesselName());
        } else {
            System.out.println("No such mode");
        }
        sc.close();
    }
}
// 3. ENTITY POJO CLASS
class NavalVessel {
    private int vesselId;
    private String vesselName;
    private int voyages;
    private String mode;
    private String vesselClass;

    public NavalVessel(int vesselId, String vesselName, int voyages,
            String mode, String vesselClass) {
        this.vesselId = vesselId;
        this.vesselName = vesselName;
        this.voyages = voyages;
        this.mode = mode;
        this.vesselClass = vesselClass;
    }

    public int getVesselId() { return vesselId; }
    public void setVesselId(int vesselId) { this.vesselId = vesselId; }
    public String getVesselName() { return vesselName; }
```

```
    public void setVesselName(String vesselName) { this.vesselName = vesselName; }
    public int getVoyages() { return voyages; }
    public void setVoyages(int voyages) { this.voyages = voyages; }
    public String getMode() { return mode; }
    public void setMode(String mode) { this.mode = mode; }
    public String getVesselClass() { return vesselClass; }
    public void setVesselClass(String vesselClass) { this.vesselClass = vesselClass; }
}
```

Problem 14: SIM Card Management - Problem Statement and I/O

1. Entity attributes (class Sim)

Class Sim with private attributes: int simId, String customerName, double balance, double ratePerSecond, String circle. Constructor plus getters/setters.

2. Static business methods (class Solution)

`transferBalance(Sim[] arr, int fromId, int toId, double amount)`
Moves amount from one SIM to another. Returns true only if both SIMs exist, they are different and the sender has balance >= amount; otherwise return false and change nothing.

`changeCircle(Sim[] arr, int simId, String newCircle)`
Updates the circle of the SIM with the given ID. Returns true if found, else false.

`sortSimsByBalanceDesc(Sim[] arr)`
Returns a NEW array of all SIMs sorted by balance in DESCENDING order (the original array is left untouched).

3. Main execution rules

Read 4 SIMs, then fromId, toId, amount, then simId and newCircle. Print "Transfer successful" or "Transfer failed". Print "Circle updated" or "Sim not found". Finally print each SIM from the sorted array as: id name balance.

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - receiver and SIM id do not exist

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
1 Amit 500.0 0.5 Kolkata 2 Raj 300.0 0.4 Delhi 3 Sita 800.0 0.6 Mumbai 4 Neha 150.0 0.3 Delhi 3 4 200 2 Chennai	Transfer successful Circle updated 3 Sita 600.0 1 Amit 500.0 4 Neha 350.0 2 Raj 300.0	1 Amit 500.0 0.5 Kolkata 2 Raj 300.0 0.4 Delhi 3 Sita 800.0 0.6 Mumbai 4 Neha 150.0 0.3 Delhi 4 9 50 7 Chennai	Transfer failed Sim not found 3 Sita 800.0 1 Amit 500.0 2 Raj 300.0 4 Neha 150.0

Explanation: Sita (800) sends 200 to Neha (150): balances become 600 and 350. Raj's circle changes to Chennai. Sorted by balance: Sita 600, Amit 500, Neha 350, Raj 300.

Exam strategy: Check every failure condition BEFORE changing any balance, so a failed transfer never leaves half-updated data.

Problem 14: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the Sim POJO. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // helper: find a SIM by id (null if missing)
    private static Sim findById(Sim[] arr, int id) {
        for (Sim s : arr) {
            if (s.getSimId() == id) return s;
        }
        return null;
    }

    // 1. STATIC BUSINESS METHODS FIRST
    public static boolean transferBalance(Sim[] arr, int fromId, int toId, double amount) {
        Sim from = findById(arr, fromId);
        Sim to = findById(arr, toId);
        if (from == null || to == null || from == to) return false;
        if (amount <= 0 || from.getBalance() < amount) return false;
        from.setBalance(from.getBalance() - amount);
        to.setBalance(to.getBalance() + amount);
        return true;
    }

    public static boolean changeCircle(Sim[] arr, int simId, String newCircle) {
        Sim s = findById(arr, simId);
        if (s == null) return false;
        s.setCircle(newCircle);
        return true;
    }
    public static Sim[] sortSimsByBalanceDesc(Sim[] arr) {
        Sim[] copy = Arrays.copyOf(arr, arr.length);
        Arrays.sort(copy, (a, b) -> Double.compare(b.getBalance(), a.getBalance()));
        return copy;
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Sim[] sims = new Sim[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt();
            sc.nextLine();
            String name = sc.nextLine();
            double balance = sc.nextDouble();
            double rate = sc.nextDouble();
            sc.nextLine();
            String circle = sc.nextLine();
            sims[i] = new Sim(id, name, balance, rate, circle);
        }
        int fromId = sc.nextInt();
        int toId = sc.nextInt();
        double amount = sc.nextDouble();
        int changeId = sc.nextInt();
        sc.nextLine();
        String newCircle = sc.nextLine();

        if (transferBalance(sims, fromId, toId, amount)) System.out.println("Transfer successful");
        else System.out.println("Transfer failed");
        if (changeCircle(sims, changeId, newCircle)) System.out.println("Circle updated");
        else System.out.println("Sim not found");

        for (Sim s : sortSimsByBalanceDesc(sims)) {
            System.out.println(s.getSimId() + " " + s.getCustomerName() + " " + s.getBalance());
        }
        sc.close();
    }
}
// 3. ENTITY POJO CLASS
class Sim {
    private int simId;
    private String customerName;
    private double balance;
    private double ratePerSecond;
    private String circle;

    public Sim(int simId, String customerName, double balance,
            double ratePerSecond, String circle) {
```

```
        this.simId = simId;
        this.customerName = customerName;
        this.balance = balance;
        this.ratePerSecond = ratePerSecond;
        this.circle = circle;
    }

    public int getSimId() { return simId; }
    public void setSimId(int simId) { this.simId = simId; }
    public String getCustomerName() { return customerName; }
    public void setCustomerName(String customerName) { this.customerName = customerName; }
    public double getBalance() { return balance; }
    public void setBalance(double balance) { this.balance = balance; }
    public double getRatePerSecond() { return ratePerSecond; }
    public void setRatePerSecond(double ratePerSecond) { this.ratePerSecond = ratePerSecond; }
    public String getCircle() { return circle; }
    public void setCircle(String circle) { this.circle = circle; }
}
```

Problem 15: Footwear Store - Problem Statement and I/O

1. Entity attributes (class Footwear)

Class Footwear with private attributes: int footwearId, String footwearName, String footwearType, int price. Constructor plus getters/setters.

2. Static business methods (class Solution)

`countFootwearByType(Footwear[] arr, String type)`
Returns the count of footwear of the given type (case-insensitive). If none, return 0.

`getSecondHighestPriceByName(Footwear[] arr, String name)`
Among footwear with the given name (case-insensitive) returns the one with the SECOND HIGHEST distinct price. If there is no second distinct price, return null.

3. Main execution rules

Read 4 footwear items, then type and name. Print the count (or "No footwear of the given type"). Then print the footwear's ID and price on separate lines (or "No such footwear").

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - unknown type, only one Bata item

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
1 Nike Sports 5000 2 Bata Formal 2000 3 Nike Casual 4000 4 Nike Sports 6000 Sports Nike	2 1 5000	1 Nike Sports 5000 2 Bata Formal 2000 3 Nike Casual 4000 4 Nike Sports 6000 Sandals Bata	No footwear of the given type No such footwear

Explanation: Two items are of type Sports (IDs 1 and 4). Nike prices are 6000, 5000, 4000, so the second highest is 5000 (ID 1).

Exam strategy: Filter by name FIRST into a list, then sort that list descending and skip entries equal to the top price.

Problem 15: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the Footwear POJO. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHODS FIRST
    public static int countFootwearByType(Footwear[] arr, String type) {
        int count = 0;
        for (Footwear f : arr) {
            if (f.getFootwearType().equalsIgnoreCase(type)) {
                count++;
            }
        }
        return count;
    }

    public static Footwear getSecondHighestPriceByName(Footwear[] arr, String name) {
        List<Footwear> list = new ArrayList<>();
        for (Footwear f : arr) {
            if (f.getFootwearName().equalsIgnoreCase(name)) {
                list.add(f);
            }
        }
        if (list.isEmpty()) return null;
        list.sort((a, b) -> b.getPrice() - a.getPrice());          // descending
        int top = list.get(0).getPrice();
        for (Footwear f : list) {
            if (f.getPrice() < top) {
                return f;
            }
        }
        return null;
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Footwear[] items = new Footwear[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt();
            sc.nextLine();
            String name = sc.nextLine();
            String type = sc.nextLine();
            int price = sc.nextInt();
            sc.nextLine();
            items[i] = new Footwear(id, name, type, price);
        }
        String searchType = sc.nextLine();
        String searchName = sc.nextLine();

        int count = countFootwearByType(items, searchType);
        if (count > 0) System.out.println(count);
        else System.out.println("No footwear of the given type");

        Footwear res = getSecondHighestPriceByName(items, searchName);
        if (res != null) {
            System.out.println(res.getFootwearId());
            System.out.println(res.getPrice());
        } else {
            System.out.println("No such footwear");
        }
        sc.close();
    }
}

// 3. ENTITY POJO CLASS
class Footwear {
    private int footwearId;
    private String footwearName;
    private String footwearType;
    private int price;
    public Footwear(int footwearId, String footwearName, String footwearType, int price) {
        this.footwearId = footwearId;
        this.footwearName = footwearName;
        this.footwearType = footwearType;
        this.price = price;
    }

    public int getFootwearId() { return footwearId; }
    public void setFootwearId(int footwearId) { this.footwearId = footwearId; }
```

```
    public String getFootwearName() { return footwearName; }
    public void setFootwearName(String footwearName) { this.footwearName = footwearName; }
    public String getFootwearType() { return footwearType; }
    public void setFootwearType(String footwearType) { this.footwearType = footwearType; }
    public int getPrice() { return price; }
    public void setPrice(int price) { this.price = price; }
}
```

Part 2: Class Association / Object Aggregation (35-Mark Pattern)

Part 1 (pages 1-43) covered single-class problems. Part 2 adds the 15 aggregation scenarios from your practice set (A01-A15), each with a full problem statement, sample and edge-case I/O, and complete code, plus 5 bonus variations (A16-A20) that use different return shapes: single MAX object, COUNT and a different filter mix. Every program was run on the sample AND on a no-match edge case; the outputs printed are the real program outputs.

What is aggregation?

Aggregation is a HAS-A relationship: the outer class keeps a reference to an inner-class object as one of its private fields. In the exam you build the inner object first, pass it to the outer constructor, and then reach inner data by chaining getters: `vehicle.getInsuranceDetails().getPremium()`.

Input reading order used in every problem

```
for each of the 4 records:
    outer field 1           (int  -> nextInt()  then sc.nextLine())
    outer field 2           (String -> nextLine())
    outer field 3           (double -> nextDouble() then sc.nextLine())
    inner field 1, 2, 3     (same rules)
    new Inner(...)  ->  new Outer(f1, f2, f3, innerObj)
after the 4 records: read the filter parameters (String via nextLine())
```

Five-step recipe (works for all 20 problems)

1. Write the inner POJO first, then the outer POJO whose last field is the inner type.

2. In the method: loop, take `Inner d = o.getXDetails();`, test the condition, add matches to a List.

3. If the list is empty return null (or 0 for a count). Never print inside the method.

4. Sort with Integer.compare / Double.compare (swap a and b for descending). Convert with toArray(new Outer[0]).

5. In main print the exact message the question asks for when the result is null.

Common mistakes that lose marks

- Forgetting sc.nextLine() after nextInt()/nextDouble() - the next String read becomes empty.

- Using == for Strings instead of equalsIgnoreCase().

- Testing the attribute on the wrong object (outer vs inner). Re-read the rule and ask who OWNS the attribute.

- Mixing up >= with > or <= with <; the rule wording decides (see A03 and A09).

- Sorting doubles with a subtraction cast to int - always use Double.compare().

- Printing the inner object reference instead of its attributes.

Problem index for Part 2

#	Domain	Method	Result	Page
A01	Vehicle & Insurance Registration	findVehicleByInsurancePlan	sorted array	45
A02	Employee & Project Allocation	findEmployeeByProjectDuration	sorted array	48
A03	Student & Scholarship Profile	getStudentsWithHighGrant	sorted array	51
A04	Product & Vendor Logistics	findProductByVendorRating	sorted array	54
A05	Hotel Room & Reservation Details	findRoomsByReservationStatus	sorted array	57
A06	Course & Faculty Management	findCourseByFacultyExperience	sorted array	60
A07	Property & Owner Records	findPropertyByOwnerStatus	sorted array	63
A08	Smart Device & Warranty Plan	findDevicesWithComprehensiveWarranty	sorted array	66
A09	Order & Shipment Profiles	findOrdersByDeliverySpeed	sorted array	69
A10	Book & Publisher Specifications	findBooksByPublisherCountry	sorted array	72
A11	Movie & Director Affiliation	findMovieByDirectorExperience	sorted array	75
A12	Flight & Aircraft Fleet	findFlightsByCapacity	sorted array	78
A13	Medical Prescription & Doctor Logs	findPrescriptionBySpecialization	sorted array	81
A14	Subscription & Account Tier	findSubscribersWithHighUsageTier	sorted array	84
A15	Inventory Item & Warehouse Location	findItemsInSafeZones	sorted array	87
A16	Bank Account & Branch (single MAX object)	findAccountWithMaxBalanceByCity	single object	90
A17	Customer & Address (COUNT result)	countCustomersByCity	int count	93
A18	Team & Coach Performance	findTeamsByCoachWinRate	sorted array	96
A19	Tablet & Display (MAX by inner attribute)	findTabletWithLargestDisplay	single object	99
A20	Library Member & Membership Plan	getMembersByPlan	sorted array	102

Problem A01: Vehicle & Insurance Registration - Statement and I/O

1. Entity classes (Aggregation: the outer class holds an object of the inner class)

Class	Private attributes (in this order)	Role
Insurance	int insuranceId, String planType, double premium	Inner (aggregated) entity
Vehicle	int vehicleId, String brand, double price, **Insurance insuranceDetails**	Outer entity: HAS-A Insurance

Each class needs a parameterized constructor and a getter and setter for every attribute. The outer constructor takes the inner object as its LAST argument.

2. Static business method (class Solution)

`Vehicle[] findVehicleByInsurancePlan(Vehicle[] arr, String targetPlanType, double minPrice)`

Filter vehicles matching targetPlanType (case-insensitive) AND price >= minPrice. Sort ascending by the insurance premium. If nothing matches, return null.

3. Main execution rules

Read 4 Vehicle objects. For each one, read the outer values (vehicleId, brand, price) then the inner values (insuranceId, planType, premium), build the Insurance object, then build the Vehicle. After the 4 objects read: targetPlanType, minPrice. Print vehicleId, brand, price of every result object on one line (space separated), or "No matching elements found." when the result is null.

Test runs (real output of the code on the next pages) - left: sample; right: edge case with no match

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
101 Honda 850000.0 1 Comprehensive 22000.0 102 Toyota 1200000.0 2 ThirdParty 9000.0 103 Hyundai 950000.0 3 comprehensive 18000.0 104 Suzuki 600000.0 4 Comprehensive 15000.0 Comprehensive 800000.0	103 Hyundai 950000.0 101 Honda 850000.0	101 Honda 850000.0 1 Comprehensive 22000.0 102 Toyota 1200000.0 2 ThirdParty 9000.0 103 Hyundai 950000.0 3 comprehensive 18000.0 104 Suzuki 600000.0 4 Comprehensive 15000.0 Platinum 500000.0	No matching elements found.

Explanation: Filter vehicles matching targetPlanType (case-insensitive) AND price >= minPrice. Sort ascending by the insurance premium. Records that pass the filter are printed in this order of vehicleId: 103, 101.

Exam strategy: Sort key lives in the INNER object: a.getInsuranceDetails().getPremium(). Vehicle 104 is dropped by the price rule, vehicle 102 by the plan type.

Problem A01: Complete Uncut Java Implementation

Complete file: Solution class (method + main) followed by the inner and outer POJO classes. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHOD FIRST
    public static Vehicle[] findVehicleByInsurancePlan(Vehicle[] arr, String targetPlanType, double minPrice) {
        List<Vehicle> list = new ArrayList<>();
        for (Vehicle o : arr) {
            Insurance d = o.getInsuranceDetails(); // inner object via aggregation
            if (d.getPlanType().equalsIgnoreCase(targetPlanType) && o.getPrice() >= minPrice) {
                list.add(o);
            }
        }
        if (list.isEmpty()) return null;
        // ascending by premium
        list.sort((a, b) ->
                Double.compare(
                        a.getInsuranceDetails().getPremium(),
                        b.getInsuranceDetails().getPremium()));
        return list.toArray(new Vehicle[0]);
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Vehicle[] arr = new Vehicle[4];
        for (int i = 0; i < 4; i++) {
            int vehicleId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String brand = sc.nextLine();
            double price = sc.nextDouble();
            sc.nextLine(); // flush newline buffer
            // ---- inner (aggregated) object attributes ----
            int insuranceId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String planType = sc.nextLine();
            double premium = sc.nextDouble();
            sc.nextLine(); // flush newline buffer
            Insurance insuranceDetailsObj = new Insurance(insuranceId, planType, premium);
            arr[i] = new Vehicle(vehicleId, brand, price, insuranceDetailsObj);
        }
        // ---- filter parameters ----
        String targetPlanType = sc.nextLine();
        double minPrice = sc.nextDouble();

        Vehicle[] result = findVehicleByInsurancePlan(arr, targetPlanType, minPrice);
        if (result != null) {
            for (Vehicle o : result) {
                System.out.println(o.getVehicleId() + " " + o.getBrand() + " " + o.getPrice());
            }
        } else {
            System.out.println("No matching elements found.");
        }
        sc.close();
    }
}

// 3. INNER (AGGREGATED) ENTITY POJO
class Insurance {
    private int insuranceId;
    private String planType;
    private double premium;

    public Insurance(int insuranceId, String planType, double premium) {
        this.insuranceId = insuranceId;
        this.planType = planType;
        this.premium = premium;
    }

    public int getInsuranceId() { return insuranceId; }
    public void setInsuranceId(int insuranceId) { this.insuranceId = insuranceId; }
    public String getPlanType() { return planType; }
    public void setPlanType(String planType) { this.planType = planType; }
    public double getPremium() { return premium; }
    public void setPremium(double premium) { this.premium = premium; }
}

// 4. OUTER ENTITY POJO (HAS-A inner object)
class Vehicle {
    private int vehicleId;
    private String brand;
    private double price;
    private Insurance insuranceDetails;

    public Vehicle(int vehicleId, String brand, double price,
            Insurance insuranceDetails) {
        this.vehicleId = vehicleId;
        this.brand = brand;
        this.price = price;
        this.insuranceDetails = insuranceDetails;
    }
```

```
    public int getVehicleId() { return vehicleId; }
    public void setVehicleId(int vehicleId) { this.vehicleId = vehicleId; }
    public String getBrand() { return brand; }
    public void setBrand(String brand) { this.brand = brand; }
    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }
    public Insurance getInsuranceDetails() { return insuranceDetails; }
    public void setInsuranceDetails(Insurance insuranceDetails) { this.insuranceDetails = insuranceDetails; }
}
```

Problem A02: Employee & Project Allocation - Statement and I/O

1. Entity classes (Aggregation: the outer class holds an object of the inner class)

Class	Private attributes (in this order)	Role
Project	int projectId, String projectName, int durationMonths	Inner (aggregated) entity
Employee	int employeeId, String name, String designation, **Project allocatedProject**	Outer entity: HAS-A Project

Each class needs a parameterized constructor and a getter and setter for every attribute. The outer constructor takes the inner object as its LAST argument.

2. Static business method (class Solution)

`Employee[] findEmployeeByProjectDuration(Employee[] arr, String targetDesignation, int minDuration)`

Filter employees with matching designation AND allocatedProject duration >= minDuration. Sort descending by employeeId. If nothing matches, return null.

3. Main execution rules

Read 4 Employee objects. For each one, read the outer values (employeeId, name, designation) then the inner values (projectId, projectName, durationMonths), build the Project object, then build the Employee. After the 4 objects read: targetDesignation, minDuration. Print employeeId, name, designation of every result object on one line (space separated), or "No matching elements found." when the result is null.

Test runs (real output of the code on the next pages) - left: sample; right: edge case with no match

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
11 Arun Developer 1 Portal 12 12 Bela Tester 2 Mobile 8 13 Chirag Developer 3 Cloud 6 14 Divya Developer 4 Banking 18 Developer 10	14 Divya Developer 11 Arun Developer	11 Arun Developer 1 Portal 12 12 Bela Tester 2 Mobile 8 13 Chirag Developer 3 Cloud 6 14 Divya Developer 4 Banking 18 Manager 5	No matching elements found.

Explanation: Filter employees with matching designation AND allocatedProject duration >= minDuration. Sort descending by employeeId. Records that pass the filter are printed in this order of employeeId: 14, 11.

Exam strategy: Mixed condition: one test on the outer object, one on the inner object. Descending order = swap a and b in the comparator.

Problem A02: Complete Uncut Java Implementation

Complete file: Solution class (method + main) followed by the inner and outer POJO classes. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHOD FIRST
    public static Employee[] findEmployeeByProjectDuration(Employee[] arr, String targetDesignation, int minDuration) {
        List<Employee> list = new ArrayList<>();
        for (Employee o : arr) {
            Project d = o.getAllocatedProject(); // inner object via aggregation
            if (o.getDesignation().equalsIgnoreCase(targetDesignation) && d.getDurationMonths() >= minDuration) {
                list.add(o);
            }
        }
        if (list.isEmpty()) return null;
        // descending by employeeId
        list.sort((a, b) ->
                Integer.compare(b.getEmployeeId(), a.getEmployeeId()));
        return list.toArray(new Employee[0]);
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Employee[] arr = new Employee[4];
        for (int i = 0; i < 4; i++) {
            int employeeId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String name = sc.nextLine();
            String designation = sc.nextLine();
            // ---- inner (aggregated) object attributes ----
            int projectId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String projectName = sc.nextLine();
            int durationMonths = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            Project allocatedProjectObj = new Project(projectId, projectName, durationMonths);
            arr[i] = new Employee(employeeId, name, designation, allocatedProjectObj);
        }
        // ---- filter parameters ----
        String targetDesignation = sc.nextLine();
        int minDuration = sc.nextInt();

        Employee[] result = findEmployeeByProjectDuration(arr, targetDesignation, minDuration);
        if (result != null) {
            for (Employee o : result) {
                System.out.println(o.getEmployeeId() + " " + o.getName() + " " + o.getDesignation());
            }
        } else {
            System.out.println("No matching elements found.");
        }
        sc.close();
    }
}

// 3. INNER (AGGREGATED) ENTITY POJO
class Project {
    private int projectId;
    private String projectName;
    private int durationMonths;

    public Project(int projectId, String projectName, int durationMonths) {
        this.projectId = projectId;
        this.projectName = projectName;
        this.durationMonths = durationMonths;
    }

    public int getProjectId() { return projectId; }
    public void setProjectId(int projectId) { this.projectId = projectId; }
    public String getProjectName() { return projectName; }
    public void setProjectName(String projectName) { this.projectName = projectName; }
    public int getDurationMonths() { return durationMonths; }
    public void setDurationMonths(int durationMonths) { this.durationMonths = durationMonths; }
}

// 4. OUTER ENTITY POJO (HAS-A inner object)
class Employee {
    private int employeeId;
    private String name;
    private String designation;
    private Project allocatedProject;

    public Employee(int employeeId, String name, String designation,
            Project allocatedProject) {
        this.employeeId = employeeId;
        this.name = name;
        this.designation = designation;
        this.allocatedProject = allocatedProject;
    }

    public int getEmployeeId() { return employeeId; }
    public void setEmployeeId(int employeeId) { this.employeeId = employeeId; }
```

```
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getDesignation() { return designation; }
    public void setDesignation(String designation) { this.designation = designation; }
    public Project getAllocatedProject() { return allocatedProject; }
    public void setAllocatedProject(Project allocatedProject) { this.allocatedProject = allocatedProject; }
}
```

Problem A03: Student & Scholarship Profile - Statement and I/O

1. Entity classes (Aggregation: the outer class holds an object of the inner class)

Class	Private attributes (in this order)	Role
Scholarship	int scholarshipId, String schemeName, double grantAmount	Inner (aggregated) entity
Student	int studentId, String studentName, double gpa, **Scholarship scholarshipDetails**	Outer entity: HAS-A Scholarship

Each class needs a parameterized constructor and a getter and setter for every attribute. The outer constructor takes the inner object as its LAST argument.

2. Static business method (class Solution)

`Student[] getStudentsWithHighGrant(Student[] arr, String targetScheme, double thresholdGrant)`

Filter students where schemeName matches targetScheme AND grantAmount > thresholdGrant (strictly greater). Sort ascending by gpa. If nothing matches, return null.

3. Main execution rules

Read 4 Student objects. For each one, read the outer values (studentId, studentName, gpa) then the inner values (scholarshipId, schemeName, grantAmount), build the Scholarship object, then build the Student. After the 4 objects read: targetScheme, thresholdGrant. Print studentId, studentName, gpa of every result object on one line (space separated), or "No matching elements found." when the result is null.

Test runs (real output of the code on the next pages) - left: sample; right: edge case with no match

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
201 Riya 8.9 1 Merit 50000.0 202 Sam 9.4 2 Merit 80000.0 203 Tara 7.8 3 Need 60000.0 204 Uday 8.2 4 Merit 45000.0 Merit 40000.0	204 Uday 8.2 201 Riya 8.9 202 Sam 9.4	201 Riya 8.9 1 Merit 50000.0 202 Sam 9.4 2 Merit 80000.0 203 Tara 7.8 3 Need 60000.0 204 Uday 8.2 4 Merit 45000.0 Sports 1000.0	No matching elements found.

Explanation: Filter students where schemeName matches targetScheme AND grantAmount > thresholdGrant (strictly greater). Sort ascending by gpa. Records that pass the filter are printed in this order of studentId: 204, 201, 202.

Exam strategy: Watch the strict '>' here (not >=). Student 203 fails on scheme, even though its grant is large.

Problem A03: Complete Uncut Java Implementation

Complete file: Solution class (method + main) followed by the inner and outer POJO classes. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHOD FIRST
    public static Student[] getStudentsWithHighGrant(Student[] arr, String targetScheme, double thresholdGrant) {
        List<Student> list = new ArrayList<>();
        for (Student o : arr) {
            Scholarship d = o.getScholarshipDetails(); // inner object via aggregation
            if (d.getSchemeName().equalsIgnoreCase(targetScheme) && d.getGrantAmount() > thresholdGrant) {
                list.add(o);
            }
        }
        if (list.isEmpty()) return null;
        // ascending by gpa
        list.sort((a, b) ->
                Double.compare(a.getGpa(), b.getGpa()));
        return list.toArray(new Student[0]);
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Student[] arr = new Student[4];
        for (int i = 0; i < 4; i++) {
            int studentId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String studentName = sc.nextLine();
            double gpa = sc.nextDouble();
            sc.nextLine(); // flush newline buffer
            // ---- inner (aggregated) object attributes ----
            int scholarshipId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String schemeName = sc.nextLine();
            double grantAmount = sc.nextDouble();
            sc.nextLine(); // flush newline buffer
            Scholarship scholarshipDetailsObj = new Scholarship(scholarshipId, schemeName, grantAmount);
            arr[i] = new Student(studentId, studentName, gpa, scholarshipDetailsObj);
        }
        // ---- filter parameters ----
        String targetScheme = sc.nextLine();
        double thresholdGrant = sc.nextDouble();

        Student[] result = getStudentsWithHighGrant(arr, targetScheme, thresholdGrant);
        if (result != null) {
            for (Student o : result) {
                System.out.println(o.getStudentId() + " " + o.getStudentName() + " " + o.getGpa());
            }
        } else {
            System.out.println("No matching elements found.");
        }
        sc.close();
    }
}

// 3. INNER (AGGREGATED) ENTITY POJO
class Scholarship {
    private int scholarshipId;
    private String schemeName;
    private double grantAmount;

    public Scholarship(int scholarshipId, String schemeName, double grantAmount) {
        this.scholarshipId = scholarshipId;
        this.schemeName = schemeName;
        this.grantAmount = grantAmount;
    }

    public int getScholarshipId() { return scholarshipId; }
    public void setScholarshipId(int scholarshipId) { this.scholarshipId = scholarshipId; }
    public String getSchemeName() { return schemeName; }
    public void setSchemeName(String schemeName) { this.schemeName = schemeName; }
    public double getGrantAmount() { return grantAmount; }
    public void setGrantAmount(double grantAmount) { this.grantAmount = grantAmount; }
}

// 4. OUTER ENTITY POJO (HAS-A inner object)
class Student {
    private int studentId;
    private String studentName;
    private double gpa;
    private Scholarship scholarshipDetails;

    public Student(int studentId, String studentName, double gpa,
            Scholarship scholarshipDetails) {
        this.studentId = studentId;
        this.studentName = studentName;
        this.gpa = gpa;
        this.scholarshipDetails = scholarshipDetails;
    }

    public int getStudentId() { return studentId; }
```

```
    public void setStudentId(int studentId) { this.studentId = studentId; }
    public String getStudentName() { return studentName; }
    public void setStudentName(String studentName) { this.studentName = studentName; }
    public double getGpa() { return gpa; }
    public void setGpa(double gpa) { this.gpa = gpa; }
    public Scholarship getScholarshipDetails() { return scholarshipDetails; }
    public void setScholarshipDetails(Scholarship scholarshipDetails) { this.scholarshipDetails = scholarshipDetails; }
}
```

Problem A04: Product & Vendor Logistics - Statement and I/O

1. Entity classes (Aggregation: the outer class holds an object of the inner class)

Class	Private attributes (in this order)	Role
Vendor	int vendorId, String vendorName, double rating	Inner (aggregated) entity
Product	int productId, String productName, int stockQuantity, **Vendor vendorDetails**	Outer entity: HAS-A Vendor

Each class needs a parameterized constructor and a getter and setter for every attribute. The outer constructor takes the inner object as its LAST argument.

2. Static business method (class Solution)

`Product[] findProductByVendorRating(Product[] arr, double minRating)`

Filter products whose vendor rating >= minRating. Sort descending by stockQuantity. If nothing matches, return null.

3. Main execution rules

Read 4 Product objects. For each one, read the outer values (productId, productName, stockQuantity) then the inner values (vendorId, vendorName, rating), build the Vendor object, then build the Product. After the 4 objects read: minRating. Print productId, productName, stockQuantity of every result object on one line (space separated), or "No matching elements found." when the result is null.

Test runs (real output of the code on the next pages) - left: sample; right: edge case with no match

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
301 Keyboard 120 1 TechSupply 4.5 302 Mouse 300 2 GadgetHub 3.8 303 Monitor 75 3 ViewMart 4.2 304 Webcam 210 4 CamWorld 4.0 4.0	304 Webcam 210 301 Keyboard 120 303 Monitor 75	301 Keyboard 120 1 TechSupply 4.5 302 Mouse 300 2 GadgetHub 3.8 303 Monitor 75 3 ViewMart 4.2 304 Webcam 210 4 CamWorld 4.0 4.9	No matching elements found.

Explanation: Filter products whose vendor rating >= minRating. Sort descending by stockQuantity. Records that pass the filter are printed in this order of productId: 304, 301, 303.

Exam strategy: Only ONE filter parameter here, and it is a double: it is the last read, so no extra nextLine() is needed after it.

Problem A04: Complete Uncut Java Implementation

Complete file: Solution class (method + main) followed by the inner and outer POJO classes. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHOD FIRST
    public static Product[] findProductByVendorRating(Product[] arr, double minRating) {
        List<Product> list = new ArrayList<>();
        for (Product o : arr) {
            Vendor d = o.getVendorDetails(); // inner object via aggregation
            if (d.getRating() >= minRating) {
                list.add(o);
            }
        }
        if (list.isEmpty()) return null;
        // descending by stockQuantity
        list.sort((a, b) ->
                Integer.compare(b.getStockQuantity(), a.getStockQuantity()));
        return list.toArray(new Product[0]);
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Product[] arr = new Product[4];
        for (int i = 0; i < 4; i++) {
            int productId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String productName = sc.nextLine();
            int stockQuantity = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            // ---- inner (aggregated) object attributes ----
            int vendorId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String vendorName = sc.nextLine();
            double rating = sc.nextDouble();
            sc.nextLine(); // flush newline buffer
            Vendor vendorDetailsObj = new Vendor(vendorId, vendorName, rating);
            arr[i] = new Product(productId, productName, stockQuantity, vendorDetailsObj);
        }
        // ---- filter parameters ----
        double minRating = sc.nextDouble();

        Product[] result = findProductByVendorRating(arr, minRating);
        if (result != null) {
            for (Product o : result) {
                System.out.println(o.getProductId() + " " + o.getProductName() + " " + o.getStockQuantity());
            }
        } else {
            System.out.println("No matching elements found.");
        }
        sc.close();
    }
}

// 3. INNER (AGGREGATED) ENTITY POJO
class Vendor {
    private int vendorId;
    private String vendorName;
    private double rating;

    public Vendor(int vendorId, String vendorName, double rating) {
        this.vendorId = vendorId;
        this.vendorName = vendorName;
        this.rating = rating;
    }

    public int getVendorId() { return vendorId; }
    public void setVendorId(int vendorId) { this.vendorId = vendorId; }
    public String getVendorName() { return vendorName; }
    public void setVendorName(String vendorName) { this.vendorName = vendorName; }
    public double getRating() { return rating; }
    public void setRating(double rating) { this.rating = rating; }
}

// 4. OUTER ENTITY POJO (HAS-A inner object)
class Product {
    private int productId;
    private String productName;
    private int stockQuantity;
    private Vendor vendorDetails;

    public Product(int productId, String productName, int stockQuantity,
            Vendor vendorDetails) {
        this.productId = productId;
        this.productName = productName;
        this.stockQuantity = stockQuantity;
        this.vendorDetails = vendorDetails;
    }

    public int getProductId() { return productId; }
    public void setProductId(int productId) { this.productId = productId; }
```

```
    public String getProductName() { return productName; }
    public void setProductName(String productName) { this.productName = productName; }
    public int getStockQuantity() { return stockQuantity; }
    public void setStockQuantity(int stockQuantity) { this.stockQuantity = stockQuantity; }
    public Vendor getVendorDetails() { return vendorDetails; }
    public void setVendorDetails(Vendor vendorDetails) { this.vendorDetails = vendorDetails; }
}
```

Problem A05: Hotel Room & Reservation Details - Statement and I/O

1. Entity classes (Aggregation: the outer class holds an object of the inner class)

Class	Private attributes (in this order)	Role
Reservation	int resId, String guestName, String status	Inner (aggregated) entity
HotelRoom	int roomNumber, String roomType, double pricePerNight, **Reservation activeReservation**	Outer entity: HAS-A Reservation

Each class needs a parameterized constructor and a getter and setter for every attribute. The outer constructor takes the inner object as its LAST argument.

2. Static business method (class Solution)

`HotelRoom[] findRoomsByReservationStatus(HotelRoom[] arr, String targetStatus)`

Filter hotel rooms where the reservation status matches targetStatus (case-insensitive). Sort ascending by pricePerNight. If nothing matches, return null.

3. Main execution rules

Read 4 HotelRoom objects. For each one, read the outer values (roomNumber, roomType, pricePerNight) then the inner values (resId, guestName, status), build the Reservation object, then build the HotelRoom. After the 4 objects read: targetStatus. Print roomNumber, roomType, pricePerNight of every result object on one line (space separated), or "No matching elements found." when the result is null.

Test runs (real output of the code on the next pages) - left: sample; right: edge case with no match

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
101 Deluxe 5500.0 1 Asha Confirmed 102 Suite 9000.0 2 Bipin Cancelled 103 Standard 3200.0 3 Chitra Confirmed 104 Deluxe 6000.0 4 Dev confirmed Confirmed	103 Standard 3200.0 101 Deluxe 5500.0 104 Deluxe 6000.0	101 Deluxe 5500.0 1 Asha Confirmed 102 Suite 9000.0 2 Bipin Cancelled 103 Standard 3200.0 3 Chitra Confirmed 104 Deluxe 6000.0 4 Dev confirmed Pending	No matching elements found.

Explanation: Filter hotel rooms where the reservation status matches targetStatus (case-insensitive). Sort ascending by pricePerNight. Records that pass the filter are printed in this order of roomNumber: 103, 101, 104.

Exam strategy: Room 104 stores 'confirmed' in lower case; equalsIgnoreCase() is what lets it match.

Problem A05: Complete Uncut Java Implementation

Complete file: Solution class (method + main) followed by the inner and outer POJO classes. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHOD FIRST
    public static HotelRoom[] findRoomsByReservationStatus(HotelRoom[] arr, String targetStatus) {
        List<HotelRoom> list = new ArrayList<>();
        for (HotelRoom o : arr) {
            Reservation d = o.getActiveReservation(); // inner object via aggregation
            if (d.getStatus().equalsIgnoreCase(targetStatus)) {
                list.add(o);
            }
        }
        if (list.isEmpty()) return null;
        // ascending by pricePerNight
        list.sort((a, b) ->
                Double.compare(a.getPricePerNight(), b.getPricePerNight()));
        return list.toArray(new HotelRoom[0]);
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        HotelRoom[] arr = new HotelRoom[4];
        for (int i = 0; i < 4; i++) {
            int roomNumber = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String roomType = sc.nextLine();
            double pricePerNight = sc.nextDouble();
            sc.nextLine(); // flush newline buffer
            // ---- inner (aggregated) object attributes ----
            int resId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String guestName = sc.nextLine();
            String status = sc.nextLine();
            Reservation activeReservationObj = new Reservation(resId, guestName, status);
            arr[i] = new HotelRoom(roomNumber, roomType, pricePerNight, activeReservationObj);
        }
        // ---- filter parameters ----
        String targetStatus = sc.nextLine();

        HotelRoom[] result = findRoomsByReservationStatus(arr, targetStatus);
        if (result != null) {
            for (HotelRoom o : result) {
                System.out.println(o.getRoomNumber() + " " + o.getRoomType() + " " + o.getPricePerNight());
            }
        } else {
            System.out.println("No matching elements found.");
        }
        sc.close();
    }
}

// 3. INNER (AGGREGATED) ENTITY POJO
class Reservation {
    private int resId;
    private String guestName;
    private String status;

    public Reservation(int resId, String guestName, String status) {
        this.resId = resId;
        this.guestName = guestName;
        this.status = status;
    }

    public int getResId() { return resId; }
    public void setResId(int resId) { this.resId = resId; }
    public String getGuestName() { return guestName; }
    public void setGuestName(String guestName) { this.guestName = guestName; }
    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }
}

// 4. OUTER ENTITY POJO (HAS-A inner object)
class HotelRoom {
    private int roomNumber;
    private String roomType;
    private double pricePerNight;
    private Reservation activeReservation;

    public HotelRoom(int roomNumber, String roomType, double pricePerNight,
            Reservation activeReservation) {
        this.roomNumber = roomNumber;
        this.roomType = roomType;
        this.pricePerNight = pricePerNight;
        this.activeReservation = activeReservation;
    }

    public int getRoomNumber() { return roomNumber; }
    public void setRoomNumber(int roomNumber) { this.roomNumber = roomNumber; }
    public String getRoomType() { return roomType; }
```

```
    public void setRoomType(String roomType) { this.roomType = roomType; }
    public double getPricePerNight() { return pricePerNight; }
    public void setPricePerNight(double pricePerNight) { this.pricePerNight = pricePerNight; }
    public Reservation getActiveReservation() { return activeReservation; }
    public void setActiveReservation(Reservation activeReservation) { this.activeReservation = activeReservation; }
}
```

Problem A06: Course & Faculty Management - Statement and I/O

1. Entity classes (Aggregation: the outer class holds an object of the inner class)

Class	Private attributes (in this order)	Role
Faculty	int facultyId, String facultyName, int experienceYears	Inner (aggregated) entity
Course	int courseId, String courseName, double fees, **Faculty assignedFaculty**	Outer entity: HAS-A Faculty

Each class needs a parameterized constructor and a getter and setter for every attribute. The outer constructor takes the inner object as its LAST argument.

2. Static business method (class Solution)

`Course[] findCourseByFacultyExperience(Course[] arr, int minExperience)`

Filter courses whose assigned faculty has experienceYears >= minExperience. Sort descending by fees. If nothing matches, return null.

3. Main execution rules

Read 4 Course objects. For each one, read the outer values (courseId, courseName, fees) then the inner values (facultyId, facultyName, experienceYears), build the Faculty object, then build the Course. After the 4 objects read: minExperience. Print courseId, courseName, fees of every result object on one line (space separated), or "No matching elements found." when the result is null.

Test runs (real output of the code on the next pages) - left: sample; right: edge case with no match

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
501 Java 15000.0 1 Dr.Rao 10 502 Python 12000.0 2 Dr.Sen 4 503 AWS 22000.0 3 Dr.Das 8 504 SQL 9000.0 4 Dr.Roy 12 8	503 AWS 22000.0 501 Java 15000.0 504 SQL 9000.0	501 Java 15000.0 1 Dr.Rao 10 502 Python 12000.0 2 Dr.Sen 4 503 AWS 22000.0 3 Dr.Das 8 504 SQL 9000.0 4 Dr.Roy 12 30	No matching elements found.

Explanation: Filter courses whose assigned faculty has experienceYears >= minExperience. Sort descending by fees. Records that pass the filter are printed in this order of courseId: 503, 501, 504.

Exam strategy: Sort descending on a double: Double.compare(b.getFees(), a.getFees()).

Problem A06: Complete Uncut Java Implementation

Complete file: Solution class (method + main) followed by the inner and outer POJO classes. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHOD FIRST
    public static Course[] findCourseByFacultyExperience(Course[] arr, int minExperience) {
        List<Course> list = new ArrayList<>();
        for (Course o : arr) {
            Faculty d = o.getAssignedFaculty(); // inner object via aggregation
            if (d.getExperienceYears() >= minExperience) {
                list.add(o);
            }
        }
        if (list.isEmpty()) return null;
        // descending by fees
        list.sort((a, b) ->
                Double.compare(b.getFees(), a.getFees()));
        return list.toArray(new Course[0]);
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Course[] arr = new Course[4];
        for (int i = 0; i < 4; i++) {
            int courseId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String courseName = sc.nextLine();
            double fees = sc.nextDouble();
            sc.nextLine(); // flush newline buffer
            // ---- inner (aggregated) object attributes ----
            int facultyId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String facultyName = sc.nextLine();
            int experienceYears = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            Faculty assignedFacultyObj = new Faculty(facultyId, facultyName, experienceYears);
            arr[i] = new Course(courseId, courseName, fees, assignedFacultyObj);
        }
        // ---- filter parameters ----
        int minExperience = sc.nextInt();

        Course[] result = findCourseByFacultyExperience(arr, minExperience);
        if (result != null) {
            for (Course o : result) {
                System.out.println(o.getCourseId() + " " + o.getCourseName() + " " + o.getFees());
            }
        } else {
            System.out.println("No matching elements found.");
        }
        sc.close();
    }
}

// 3. INNER (AGGREGATED) ENTITY POJO
class Faculty {
    private int facultyId;
    private String facultyName;
    private int experienceYears;

    public Faculty(int facultyId, String facultyName, int experienceYears) {
        this.facultyId = facultyId;
        this.facultyName = facultyName;
        this.experienceYears = experienceYears;
    }

    public int getFacultyId() { return facultyId; }
    public void setFacultyId(int facultyId) { this.facultyId = facultyId; }
    public String getFacultyName() { return facultyName; }
    public void setFacultyName(String facultyName) { this.facultyName = facultyName; }
    public int getExperienceYears() { return experienceYears; }
    public void setExperienceYears(int experienceYears) { this.experienceYears = experienceYears; }
}

// 4. OUTER ENTITY POJO (HAS-A inner object)
class Course {
    private int courseId;
    private String courseName;
    private double fees;
    private Faculty assignedFaculty;

    public Course(int courseId, String courseName, double fees,
            Faculty assignedFaculty) {
        this.courseId = courseId;
        this.courseName = courseName;
        this.fees = fees;
        this.assignedFaculty = assignedFaculty;
    }

    public int getCourseId() { return courseId; }
    public void setCourseId(int courseId) { this.courseId = courseId; }
```

```
    public String getCourseName() { return courseName; }
    public void setCourseName(String courseName) { this.courseName = courseName; }
    public double getFees() { return fees; }
    public void setFees(double fees) { this.fees = fees; }
    public Faculty getAssignedFaculty() { return assignedFaculty; }
    public void setAssignedFaculty(Faculty assignedFaculty) { this.assignedFaculty = assignedFaculty; }
}
```

Problem A07: Property & Owner Records - Statement and I/O

1. Entity classes (Aggregation: the outer class holds an object of the inner class)

Class	Private attributes (in this order)	Role
Owner	int ownerId, String ownerName, String verificationStatus	Inner (aggregated) entity
Property	int propertyId, String propertyType, double marketValue, **Owner ownerDetails**	Outer entity: HAS-A Owner

Each class needs a parameterized constructor and a getter and setter for every attribute. The outer constructor takes the inner object as its LAST argument.

2. Static business method (class Solution)

`Property[] findPropertyByOwnerStatus(Property[] arr, String targetType)`

Filter properties with matching propertyType AND an owner whose verificationStatus is 'Verified'. Sort descending by marketValue. If nothing matches, return null.

3. Main execution rules

Read 4 Property objects. For each one, read the outer values (propertyId, propertyType, marketValue) then the inner values (ownerId, ownerName, verificationStatus), build the Owner object, then build the Property. After the 4 objects read: targetType. Print propertyId, propertyType, marketValue of every result object on one line (space separated), or "No matching elements found." when the result is null.

Test runs (real output of the code on the next pages) - left: sample; right: edge case with no match

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
601 Apartment 4500000.0 1 Ravi Verified 602 Villa 9500000.0 2 Sana Verified 603 Apartment 5200000.0 3 Tina Pending 604 Apartment 6100000.0 4 Umesh Verified Apartment	604 Apartment 6100000.0 601 Apartment 4500000.0	601 Apartment 4500000.0 1 Ravi Verified 602 Villa 9500000.0 2 Sana Verified 603 Apartment 5200000.0 3 Tina Pending 604 Apartment 6100000.0 4 Umesh Verified Plot	No matching elements found.

Explanation: Filter properties with matching propertyType AND an owner whose verificationStatus is 'Verified'. Sort descending by marketValue. Records that pass the filter are printed in this order of propertyId: 604, 601.

Exam strategy: 'Verified' is a fixed literal in the rule, not an input. Property 603 is an Apartment but its owner is still Pending.

Problem A07: Complete Uncut Java Implementation

Complete file: Solution class (method + main) followed by the inner and outer POJO classes. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHOD FIRST
    public static Property[] findPropertyByOwnerStatus(Property[] arr, String targetType) {
        List<Property> list = new ArrayList<>();
        for (Property o : arr) {
            Owner d = o.getOwnerDetails(); // inner object via aggregation
            if (o.getPropertyType().equalsIgnoreCase(targetType) && d.getVerificationStatus().equalsIgnoreCase("Verified")) {
                list.add(o);
            }
        }
        if (list.isEmpty()) return null;
        // descending by marketValue
        list.sort((a, b) ->
                Double.compare(b.getMarketValue(), a.getMarketValue()));
        return list.toArray(new Property[0]);
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Property[] arr = new Property[4];
        for (int i = 0; i < 4; i++) {
            int propertyId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String propertyType = sc.nextLine();
            double marketValue = sc.nextDouble();
            sc.nextLine(); // flush newline buffer
            // ---- inner (aggregated) object attributes ----
            int ownerId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String ownerName = sc.nextLine();
            String verificationStatus = sc.nextLine();
            Owner ownerDetailsObj = new Owner(ownerId, ownerName, verificationStatus);
            arr[i] = new Property(propertyId, propertyType, marketValue, ownerDetailsObj);
        }
        // ---- filter parameters ----
        String targetType = sc.nextLine();

        Property[] result = findPropertyByOwnerStatus(arr, targetType);
        if (result != null) {
            for (Property o : result) {
                System.out.println(o.getPropertyId() + " " + o.getPropertyType() + " " + o.getMarketValue());
            }
        } else {
            System.out.println("No matching elements found.");
        }
        sc.close();
    }
}

// 3. INNER (AGGREGATED) ENTITY POJO
class Owner {
    private int ownerId;
    private String ownerName;
    private String verificationStatus;

    public Owner(int ownerId, String ownerName, String verificationStatus) {
        this.ownerId = ownerId;
        this.ownerName = ownerName;
        this.verificationStatus = verificationStatus;
    }

    public int getOwnerId() { return ownerId; }
    public void setOwnerId(int ownerId) { this.ownerId = ownerId; }
    public String getOwnerName() { return ownerName; }
    public void setOwnerName(String ownerName) { this.ownerName = ownerName; }
    public String getVerificationStatus() { return verificationStatus; }
    public void setVerificationStatus(String verificationStatus) { this.verificationStatus = verificationStatus; }
}

// 4. OUTER ENTITY POJO (HAS-A inner object)
class Property {
    private int propertyId;
    private String propertyType;
    private double marketValue;
    private Owner ownerDetails;

    public Property(int propertyId, String propertyType, double marketValue,
            Owner ownerDetails) {
        this.propertyId = propertyId;
        this.propertyType = propertyType;
        this.marketValue = marketValue;
        this.ownerDetails = ownerDetails;
    }

    public int getPropertyId() { return propertyId; }
    public void setPropertyId(int propertyId) { this.propertyId = propertyId; }
    public String getPropertyType() { return propertyType; }
```

```
    public void setPropertyType(String propertyType) { this.propertyType = propertyType; }
    public double getMarketValue() { return marketValue; }
    public void setMarketValue(double marketValue) { this.marketValue = marketValue; }
    public Owner getOwnerDetails() { return ownerDetails; }
    public void setOwnerDetails(Owner ownerDetails) { this.ownerDetails = ownerDetails; }
}
```

Problem A08: Smart Device & Warranty Plan - Statement and I/O

1. Entity classes (Aggregation: the outer class holds an object of the inner class)

Class	Private attributes (in this order)	Role
Warranty	int warrantyId, int periodMonths, String coverageType	Inner (aggregated) entity
SmartDevice	int deviceId, String deviceName, String brand, **Warranty warrantyDetails**	Outer entity: HAS-A Warranty

Each class needs a parameterized constructor and a getter and setter for every attribute. The outer constructor takes the inner object as its LAST argument.

2. Static business method (class Solution)

`SmartDevice[] findDevicesWithComprehensiveWarranty(SmartDevice[] arr, String targetCoverage, int minPeriod)`

Filter devices matching coverageType AND periodMonths >= minPeriod. Sort ascending by deviceId. If nothing matches, return null.

3. Main execution rules

Read 4 SmartDevice objects. For each one, read the outer values (deviceId, deviceName, brand) then the inner values (warrantyId, periodMonths, coverageType), build the Warranty object, then build the SmartDevice. After the 4 objects read: targetCoverage, minPeriod. Print deviceId, deviceName, brand of every result object on one line (space separated), or "No matching elements found." when the result is null.

Test runs (real output of the code on the next pages) - left: sample; right: edge case with no match

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
704 Watch Apple 4 12 Comprehensive 702 Speaker Bose 2 36 Comprehensive 703 Camera Canon 3 24 Limited 701 SmartTV Sony 1 24 Comprehensive Comprehensive 18	701 SmartTV Sony 702 Speaker Bose	704 Watch Apple 4 12 Comprehensive 702 Speaker Bose 2 36 Comprehensive 703 Camera Canon 3 24 Limited 701 SmartTV Sony 1 24 Comprehensive Accidental 6	No matching elements found.

Explanation: Filter devices matching coverageType AND periodMonths >= minPeriod. Sort ascending by deviceId. Records that pass the filter are printed in this order of deviceId: 701, 702.

Exam strategy: Input order is 704, 702, 703, 701, so you can SEE the ascending sort work. Note that the inner object here is read as int, int, String.

Problem A08: Complete Uncut Java Implementation

Complete file: Solution class (method + main) followed by the inner and outer POJO classes. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHOD FIRST
    public static SmartDevice[] findDevicesWithComprehensiveWarranty(SmartDevice[] arr, String targetCoverage, int minPeriod) {
        List<SmartDevice> list = new ArrayList<>();
        for (SmartDevice o : arr) {
            Warranty d = o.getWarrantyDetails(); // inner object via aggregation
            if (d.getCoverageType().equalsIgnoreCase(targetCoverage) && d.getPeriodMonths() >= minPeriod) {
                list.add(o);
            }
        }
        if (list.isEmpty()) return null;
        // ascending by deviceId
        list.sort((a, b) ->
                Integer.compare(a.getDeviceId(), b.getDeviceId()));
        return list.toArray(new SmartDevice[0]);
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        SmartDevice[] arr = new SmartDevice[4];
        for (int i = 0; i < 4; i++) {
            int deviceId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String deviceName = sc.nextLine();
            String brand = sc.nextLine();
            // ---- inner (aggregated) object attributes ----
            int warrantyId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            int periodMonths = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String coverageType = sc.nextLine();
            Warranty warrantyDetailsObj = new Warranty(warrantyId, periodMonths, coverageType);
            arr[i] = new SmartDevice(deviceId, deviceName, brand, warrantyDetailsObj);
        }
        // ---- filter parameters ----
        String targetCoverage = sc.nextLine();
        int minPeriod = sc.nextInt();

        SmartDevice[] result = findDevicesWithComprehensiveWarranty(arr, targetCoverage, minPeriod);
        if (result != null) {
            for (SmartDevice o : result) {
                System.out.println(o.getDeviceId() + " " + o.getDeviceName() + " " + o.getBrand());
            }
        } else {
            System.out.println("No matching elements found.");
        }
        sc.close();
    }
}

// 3. INNER (AGGREGATED) ENTITY POJO
class Warranty {
    private int warrantyId;
    private int periodMonths;
    private String coverageType;

    public Warranty(int warrantyId, int periodMonths, String coverageType) {
        this.warrantyId = warrantyId;
        this.periodMonths = periodMonths;
        this.coverageType = coverageType;
    }

    public int getWarrantyId() { return warrantyId; }
    public void setWarrantyId(int warrantyId) { this.warrantyId = warrantyId; }
    public int getPeriodMonths() { return periodMonths; }
    public void setPeriodMonths(int periodMonths) { this.periodMonths = periodMonths; }
    public String getCoverageType() { return coverageType; }
    public void setCoverageType(String coverageType) { this.coverageType = coverageType; }
}

// 4. OUTER ENTITY POJO (HAS-A inner object)
class SmartDevice {
    private int deviceId;
    private String deviceName;
    private String brand;
    private Warranty warrantyDetails;

    public SmartDevice(int deviceId, String deviceName, String brand,
            Warranty warrantyDetails) {
        this.deviceId = deviceId;
        this.deviceName = deviceName;
        this.brand = brand;
        this.warrantyDetails = warrantyDetails;
    }

    public int getDeviceId() { return deviceId; }
    public void setDeviceId(int deviceId) { this.deviceId = deviceId; }
```

```
    public String getDeviceName() { return deviceName; }
    public void setDeviceName(String deviceName) { this.deviceName = deviceName; }
    public String getBrand() { return brand; }
    public void setBrand(String brand) { this.brand = brand; }
    public Warranty getWarrantyDetails() { return warrantyDetails; }
    public void setWarrantyDetails(Warranty warrantyDetails) { this.warrantyDetails = warrantyDetails; }
}
```

Problem A09: Order & Shipment Profiles - Statement and I/O

1. Entity classes (Aggregation: the outer class holds an object of the inner class)

Class	Private attributes (in this order)	Role
Shipment	int shipmentId, String carrierName, int deliveryDays	Inner (aggregated) entity
Order	int orderId, String itemCategory, double totalAmount, **Shipment shipmentDetails**	Outer entity: HAS-A Shipment

Each class needs a parameterized constructor and a getter and setter for every attribute. The outer constructor takes the inner object as its LAST argument.

2. Static business method (class Solution)

`Order[] findOrdersByDeliverySpeed(Order[] arr, String targetCategory, int maxDays)`

Filter orders matching itemCategory AND deliveryDays <= maxDays. Sort descending by totalAmount. If nothing matches, return null.

3. Main execution rules

Read 4 Order objects. For each one, read the outer values (orderId, itemCategory, totalAmount) then the inner values (shipmentId, carrierName, deliveryDays), build the Shipment object, then build the Order. After the 4 objects read: targetCategory, maxDays. Print orderId, itemCategory, totalAmount of every result object on one line (space separated), or "No matching elements found." when the result is null.

Test runs (real output of the code on the next pages) - left: sample; right: edge case with no match

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
801 Electronics 25000.0 1 BlueDart 2 802 Books 1200.0 2 DTDC 5 803 Electronics 48000.0 3 Delhivery 3 804 Electronics 15000.0 4 EcomExp 7 Electronics 4	803 Electronics 48000.0 801 Electronics 25000.0	801 Electronics 25000.0 1 BlueDart 2 802 Books 1200.0 2 DTDC 5 803 Electronics 48000.0 3 Delhivery 3 804 Electronics 15000.0 4 EcomExp 7 Toys 1	No matching elements found.

Explanation: Filter orders matching itemCategory AND deliveryDays <= maxDays. Sort descending by totalAmount. Records that pass the filter are printed in this order of orderId: 803, 801.

Exam strategy: Direction of the comparison flips here: smaller delivery time is better, so the rule is <= maxDays.

Problem A09: Complete Uncut Java Implementation

Complete file: Solution class (method + main) followed by the inner and outer POJO classes. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHOD FIRST
    public static Order[] findOrdersByDeliverySpeed(Order[] arr, String targetCategory, int maxDays) {
        List<Order> list = new ArrayList<>();
        for (Order o : arr) {
            Shipment d = o.getShipmentDetails(); // inner object via aggregation
            if (o.getItemCategory().equalsIgnoreCase(targetCategory) && d.getDeliveryDays() <= maxDays) {
                list.add(o);
            }
        }
        if (list.isEmpty()) return null;
        // descending by totalAmount
        list.sort((a, b) ->
                Double.compare(b.getTotalAmount(), a.getTotalAmount()));
        return list.toArray(new Order[0]);
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Order[] arr = new Order[4];
        for (int i = 0; i < 4; i++) {
            int orderId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String itemCategory = sc.nextLine();
            double totalAmount = sc.nextDouble();
            sc.nextLine(); // flush newline buffer
            // ---- inner (aggregated) object attributes ----
            int shipmentId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String carrierName = sc.nextLine();
            int deliveryDays = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            Shipment shipmentDetailsObj = new Shipment(shipmentId, carrierName, deliveryDays);
            arr[i] = new Order(orderId, itemCategory, totalAmount, shipmentDetailsObj);
        }
        // ---- filter parameters ----
        String targetCategory = sc.nextLine();
        int maxDays = sc.nextInt();

        Order[] result = findOrdersByDeliverySpeed(arr, targetCategory, maxDays);
        if (result != null) {
            for (Order o : result) {
                System.out.println(o.getOrderId() + " " + o.getItemCategory() + " " + o.getTotalAmount());
            }
        } else {
            System.out.println("No matching elements found.");
        }
        sc.close();
    }
}

// 3. INNER (AGGREGATED) ENTITY POJO
class Shipment {
    private int shipmentId;
    private String carrierName;
    private int deliveryDays;

    public Shipment(int shipmentId, String carrierName, int deliveryDays) {
        this.shipmentId = shipmentId;
        this.carrierName = carrierName;
        this.deliveryDays = deliveryDays;
    }

    public int getShipmentId() { return shipmentId; }
    public void setShipmentId(int shipmentId) { this.shipmentId = shipmentId; }
    public String getCarrierName() { return carrierName; }
    public void setCarrierName(String carrierName) { this.carrierName = carrierName; }
    public int getDeliveryDays() { return deliveryDays; }
    public void setDeliveryDays(int deliveryDays) { this.deliveryDays = deliveryDays; }
}

// 4. OUTER ENTITY POJO (HAS-A inner object)
class Order {
    private int orderId;
    private String itemCategory;
    private double totalAmount;
    private Shipment shipmentDetails;

    public Order(int orderId, String itemCategory, double totalAmount,
            Shipment shipmentDetails) {
        this.orderId = orderId;
        this.itemCategory = itemCategory;
        this.totalAmount = totalAmount;
        this.shipmentDetails = shipmentDetails;
    }

    public int getOrderId() { return orderId; }
```

```
    public void setOrderId(int orderId) { this.orderId = orderId; }
    public String getItemCategory() { return itemCategory; }
    public void setItemCategory(String itemCategory) { this.itemCategory = itemCategory; }
    public double getTotalAmount() { return totalAmount; }
    public void setTotalAmount(double totalAmount) { this.totalAmount = totalAmount; }
    public Shipment getShipmentDetails() { return shipmentDetails; }
    public void setShipmentDetails(Shipment shipmentDetails) { this.shipmentDetails = shipmentDetails; }
}
```

Problem A10: Book & Publisher Specifications - Statement and I/O

1. Entity classes (Aggregation: the outer class holds an object of the inner class)

Class	Private attributes (in this order)	Role
Publisher	int publisherId, String publisherName, String country	Inner (aggregated) entity
Book	int bookId, String title, double price, **Publisher publisherDetails**	Outer entity: HAS-A Publisher

Each class needs a parameterized constructor and a getter and setter for every attribute. The outer constructor takes the inner object as its LAST argument.

2. Static business method (class Solution)

`Book[] findBooksByPublisherCountry(Book[] arr, String targetCountry)`

Filter books where the publisher country matches targetCountry. Sort ascending by price. If nothing matches, return null.

3. Main execution rules

Read 4 Book objects. For each one, read the outer values (bookId, title, price) then the inner values (publisherId, publisherName, country), build the Publisher object, then build the Book. After the 4 objects read: targetCountry. Print bookId, title, price of every result object on one line (space separated), or "No matching elements found." when the result is null.

Test runs (real output of the code on the next pages) - left: sample; right: edge case with no match

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
901 JavaGuide 650.0 1 McGrawHill USA 902 Wings 300.0 2 Rupa India 903 Algorithms 890.0 3 Pearson USA 904 Networks 450.0 4 Wiley USA USA	904 Networks 450.0 901 JavaGuide 650.0 903 Algorithms 890.0	901 JavaGuide 650.0 1 McGrawHill USA 902 Wings 300.0 2 Rupa India 903 Algorithms 890.0 3 Pearson USA 904 Networks 450.0 4 Wiley USA Japan	No matching elements found.

Explanation: Filter books where the publisher country matches targetCountry. Sort ascending by price. Records that pass the filter are printed in this order of bookId: 904, 901, 903.

Exam strategy: The simplest pattern of the set: one inner-object filter, one outer-object sort key.

Problem A10: Complete Uncut Java Implementation

Complete file: Solution class (method + main) followed by the inner and outer POJO classes. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHOD FIRST
    public static Book[] findBooksByPublisherCountry(Book[] arr, String targetCountry) {
        List<Book> list = new ArrayList<>();
        for (Book o : arr) {
            Publisher d = o.getPublisherDetails(); // inner object via aggregation
            if (d.getCountry().equalsIgnoreCase(targetCountry)) {
                list.add(o);
            }
        }
        if (list.isEmpty()) return null;
        // ascending by price
        list.sort((a, b) ->
                Double.compare(a.getPrice(), b.getPrice()));
        return list.toArray(new Book[0]);
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Book[] arr = new Book[4];
        for (int i = 0; i < 4; i++) {
            int bookId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String title = sc.nextLine();
            double price = sc.nextDouble();
            sc.nextLine(); // flush newline buffer
            // ---- inner (aggregated) object attributes ----
            int publisherId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String publisherName = sc.nextLine();
            String country = sc.nextLine();
            Publisher publisherDetailsObj = new Publisher(publisherId, publisherName, country);
            arr[i] = new Book(bookId, title, price, publisherDetailsObj);
        }
        // ---- filter parameters ----
        String targetCountry = sc.nextLine();

        Book[] result = findBooksByPublisherCountry(arr, targetCountry);
        if (result != null) {
            for (Book o : result) {
                System.out.println(o.getBookId() + " " + o.getTitle() + " " + o.getPrice());
            }
        } else {
            System.out.println("No matching elements found.");
        }
        sc.close();
    }
}

// 3. INNER (AGGREGATED) ENTITY POJO
class Publisher {
    private int publisherId;
    private String publisherName;
    private String country;

    public Publisher(int publisherId, String publisherName, String country) {
        this.publisherId = publisherId;
        this.publisherName = publisherName;
        this.country = country;
    }

    public int getPublisherId() { return publisherId; }
    public void setPublisherId(int publisherId) { this.publisherId = publisherId; }
    public String getPublisherName() { return publisherName; }
    public void setPublisherName(String publisherName) { this.publisherName = publisherName; }
    public String getCountry() { return country; }
    public void setCountry(String country) { this.country = country; }
}

// 4. OUTER ENTITY POJO (HAS-A inner object)
class Book {
    private int bookId;
    private String title;
    private double price;
    private Publisher publisherDetails;

    public Book(int bookId, String title, double price,
            Publisher publisherDetails) {
        this.bookId = bookId;
        this.title = title;
        this.price = price;
        this.publisherDetails = publisherDetails;
    }

    public int getBookId() { return bookId; }
    public void setBookId(int bookId) { this.bookId = bookId; }
    public String getTitle() { return title; }
```

```
    public void setTitle(String title) { this.title = title; }
    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }
    public Publisher getPublisherDetails() { return publisherDetails; }
    public void setPublisherDetails(Publisher publisherDetails) { this.publisherDetails = publisherDetails; }
}
```

Problem A11: Movie & Director Affiliation - Statement and I/O

1. Entity classes (Aggregation: the outer class holds an object of the inner class)

Class	Private attributes (in this order)	Role
Director	int directorId, String directorName, int totalMovies	Inner (aggregated) entity
Movie	int movieId, String movieTitle, double imdbRating, **Director directorDetails**	Outer entity: HAS-A Director

Each class needs a parameterized constructor and a getter and setter for every attribute. The outer constructor takes the inner object as its LAST argument.

2. Static business method (class Solution)

`Movie[] findMovieByDirectorExperience(Movie[] arr, int minTotalMovies)`

Filter movies where the director's totalMovies >= minTotalMovies. Sort descending by imdbRating. If nothing matches, return null.

3. Main execution rules

Read 4 Movie objects. For each one, read the outer values (movieId, movieTitle, imdbRating) then the inner values (directorId, directorName, totalMovies), build the Director object, then build the Movie. After the 4 objects read: minTotalMovies. Print movieId, movieTitle, imdbRating of every result object on one line (space separated), or "No matching elements found." when the result is null.

Test runs (real output of the code on the next pages) - left: sample; right: edge case with no match

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
1001 Skyline 7.8 1 Nolan 12 1002 Echoes 8.6 2 Anand 25 1003 Drift 6.9 3 Bose 4 1004 Horizon 8.1 4 Kapoor 15 10	1002 Echoes 8.6 1004 Horizon 8.1 1001 Skyline 7.8	1001 Skyline 7.8 1 Nolan 12 1002 Echoes 8.6 2 Anand 25 1003 Drift 6.9 3 Bose 4 1004 Horizon 8.1 4 Kapoor 15 100	No matching elements found.

Explanation: Filter movies where the director's totalMovies >= minTotalMovies. Sort descending by imdbRating. Records that pass the filter are printed in this order of movieId: 1002, 1004, 1001.

Exam strategy: imdbRating is a double, so use Double.compare(); never subtract doubles and cast to int.

Problem A11: Complete Uncut Java Implementation

Complete file: Solution class (method + main) followed by the inner and outer POJO classes. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHOD FIRST
    public static Movie[] findMovieByDirectorExperience(Movie[] arr, int minTotalMovies) {
        List<Movie> list = new ArrayList<>();
        for (Movie o : arr) {
            Director d = o.getDirectorDetails(); // inner object via aggregation
            if (d.getTotalMovies() >= minTotalMovies) {
                list.add(o);
            }
        }
        if (list.isEmpty()) return null;
        // descending by imdbRating
        list.sort((a, b) ->
                Double.compare(b.getImdbRating(), a.getImdbRating()));
        return list.toArray(new Movie[0]);
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Movie[] arr = new Movie[4];
        for (int i = 0; i < 4; i++) {
            int movieId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String movieTitle = sc.nextLine();
            double imdbRating = sc.nextDouble();
            sc.nextLine(); // flush newline buffer
            // ---- inner (aggregated) object attributes ----
            int directorId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String directorName = sc.nextLine();
            int totalMovies = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            Director directorDetailsObj = new Director(directorId, directorName, totalMovies);
            arr[i] = new Movie(movieId, movieTitle, imdbRating, directorDetailsObj);
        }
        // ---- filter parameters ----
        int minTotalMovies = sc.nextInt();

        Movie[] result = findMovieByDirectorExperience(arr, minTotalMovies);
        if (result != null) {
            for (Movie o : result) {
                System.out.println(o.getMovieId() + " " + o.getMovieTitle() + " " + o.getImdbRating());
            }
        } else {
            System.out.println("No matching elements found.");
        }
        sc.close();
    }
}

// 3. INNER (AGGREGATED) ENTITY POJO
class Director {
    private int directorId;
    private String directorName;
    private int totalMovies;

    public Director(int directorId, String directorName, int totalMovies) {
        this.directorId = directorId;
        this.directorName = directorName;
        this.totalMovies = totalMovies;
    }

    public int getDirectorId() { return directorId; }
    public void setDirectorId(int directorId) { this.directorId = directorId; }
    public String getDirectorName() { return directorName; }
    public void setDirectorName(String directorName) { this.directorName = directorName; }
    public int getTotalMovies() { return totalMovies; }
    public void setTotalMovies(int totalMovies) { this.totalMovies = totalMovies; }
}

// 4. OUTER ENTITY POJO (HAS-A inner object)
class Movie {
    private int movieId;
    private String movieTitle;
    private double imdbRating;
    private Director directorDetails;

    public Movie(int movieId, String movieTitle, double imdbRating,
            Director directorDetails) {
        this.movieId = movieId;
        this.movieTitle = movieTitle;
        this.imdbRating = imdbRating;
        this.directorDetails = directorDetails;
    }

    public int getMovieId() { return movieId; }
    public void setMovieId(int movieId) { this.movieId = movieId; }
```

```
    public String getMovieTitle() { return movieTitle; }
    public void setMovieTitle(String movieTitle) { this.movieTitle = movieTitle; }
    public double getImdbRating() { return imdbRating; }
    public void setImdbRating(double imdbRating) { this.imdbRating = imdbRating; }
    public Director getDirectorDetails() { return directorDetails; }
    public void setDirectorDetails(Director directorDetails) { this.directorDetails = directorDetails; }
}
```

Problem A12: Flight & Aircraft Fleet - Statement and I/O

1. Entity classes (Aggregation: the outer class holds an object of the inner class)

Class	Private attributes (in this order)	Role
Aircraft	int aircraftId, String model, int capacity	Inner (aggregated) entity
Flight	int flightId, String destination, double ticketFare, **Aircraft aircraftDetails**	Outer entity: HAS-A Aircraft

Each class needs a parameterized constructor and a getter and setter for every attribute. The outer constructor takes the inner object as its LAST argument.

2. Static business method (class Solution)

`Flight[] findFlightsByCapacity(Flight[] arr, String targetDestination, int minCapacity)`

Filter flights matching destination AND aircraft capacity >= minCapacity. Sort ascending by ticketFare. If nothing matches, return null.

3. Main execution rules

Read 4 Flight objects. For each one, read the outer values (flightId, destination, ticketFare) then the inner values (aircraftId, model, capacity), build the Aircraft object, then build the Flight. After the 4 objects read: targetDestination, minCapacity. Print flightId, destination, ticketFare of every result object on one line (space separated), or "No matching elements found." when the result is null.

Test runs (real output of the code on the next pages) - left: sample; right: edge case with no match

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
1101 Delhi 5200.0 1 A320 180 1102 Mumbai 4800.0 2 B737 160 1103 Delhi 4500.0 3 A321 220 1104 Delhi 6100.0 4 ATR72 70 Delhi 150	1103 Delhi 4500.0 1101 Delhi 5200.0	1101 Delhi 5200.0 1 A320 180 1102 Mumbai 4800.0 2 B737 160 1103 Delhi 4500.0 3 A321 220 1104 Delhi 6100.0 4 ATR72 70 Goa 100	No matching elements found.

Explanation: Filter flights matching destination AND aircraft capacity >= minCapacity. Sort ascending by ticketFare. Records that pass the filter are printed in this order of flightId: 1103, 1101.

Exam strategy: Flight 1104 goes to Delhi but its ATR72 seats only 70, so the capacity rule removes it.

Problem A12: Complete Uncut Java Implementation

Complete file: Solution class (method + main) followed by the inner and outer POJO classes. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHOD FIRST
    public static Flight[] findFlightsByCapacity(Flight[] arr, String targetDestination, int minCapacity) {
        List<Flight> list = new ArrayList<>();
        for (Flight o : arr) {
            Aircraft d = o.getAircraftDetails(); // inner object via aggregation
            if (o.getDestination().equalsIgnoreCase(targetDestination) && d.getCapacity() >= minCapacity) {
                list.add(o);
            }
        }
        if (list.isEmpty()) return null;
        // ascending by ticketFare
        list.sort((a, b) ->
                Double.compare(a.getTicketFare(), b.getTicketFare()));
        return list.toArray(new Flight[0]);
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Flight[] arr = new Flight[4];
        for (int i = 0; i < 4; i++) {
            int flightId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String destination = sc.nextLine();
            double ticketFare = sc.nextDouble();
            sc.nextLine(); // flush newline buffer
            // ---- inner (aggregated) object attributes ----
            int aircraftId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String model = sc.nextLine();
            int capacity = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            Aircraft aircraftDetailsObj = new Aircraft(aircraftId, model, capacity);
            arr[i] = new Flight(flightId, destination, ticketFare, aircraftDetailsObj);
        }
        // ---- filter parameters ----
        String targetDestination = sc.nextLine();
        int minCapacity = sc.nextInt();

        Flight[] result = findFlightsByCapacity(arr, targetDestination, minCapacity);
        if (result != null) {
            for (Flight o : result) {
                System.out.println(o.getFlightId() + " " + o.getDestination() + " " + o.getTicketFare());
            }
        } else {
            System.out.println("No matching elements found.");
        }
        sc.close();
    }
}

// 3. INNER (AGGREGATED) ENTITY POJO
class Aircraft {
    private int aircraftId;
    private String model;
    private int capacity;

    public Aircraft(int aircraftId, String model, int capacity) {
        this.aircraftId = aircraftId;
        this.model = model;
        this.capacity = capacity;
    }

    public int getAircraftId() { return aircraftId; }
    public void setAircraftId(int aircraftId) { this.aircraftId = aircraftId; }
    public String getModel() { return model; }
    public void setModel(String model) { this.model = model; }
    public int getCapacity() { return capacity; }
    public void setCapacity(int capacity) { this.capacity = capacity; }
}

// 4. OUTER ENTITY POJO (HAS-A inner object)
class Flight {
    private int flightId;
    private String destination;
    private double ticketFare;
    private Aircraft aircraftDetails;

    public Flight(int flightId, String destination, double ticketFare,
            Aircraft aircraftDetails) {
        this.flightId = flightId;
        this.destination = destination;
        this.ticketFare = ticketFare;
        this.aircraftDetails = aircraftDetails;
    }

    public int getFlightId() { return flightId; }
```

```
    public void setFlightId(int flightId) { this.flightId = flightId; }
    public String getDestination() { return destination; }
    public void setDestination(String destination) { this.destination = destination; }
    public double getTicketFare() { return ticketFare; }
    public void setTicketFare(double ticketFare) { this.ticketFare = ticketFare; }
    public Aircraft getAircraftDetails() { return aircraftDetails; }
    public void setAircraftDetails(Aircraft aircraftDetails) { this.aircraftDetails = aircraftDetails; }
}
```

Problem A13: Medical Prescription & Doctor Logs - Statement and I/O

1. Entity classes (Aggregation: the outer class holds an object of the inner class)

Class	Private attributes (in this order)	Role
Doctor	int doctorId, String doctorName, String specialization	Inner (aggregated) entity
Prescription	int prescriptionId, String medicineName, double cost, **Doctor doctorDetails**	Outer entity: HAS-A Doctor

Each class needs a parameterized constructor and a getter and setter for every attribute. The outer constructor takes the inner object as its LAST argument.

2. Static business method (class Solution)

`Prescription[] findPrescriptionBySpecialization(Prescription[] arr, String targetSpecialization)`

Filter prescriptions where the doctor's specialization matches targetSpecialization. Sort descending by cost. If nothing matches, return null.

3. Main execution rules

Read 4 Prescription objects. For each one, read the outer values (prescriptionId, medicineName, cost) then the inner values (doctorId, doctorName, specialization), build the Doctor object, then build the Prescription. After the 4 objects read: targetSpecialization. Print prescriptionId, medicineName, cost of every result object on one line (space separated), or "No matching elements found." when the result is null.

Test runs (real output of the code on the next pages) - left: sample; right: edge case with no match

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
1201 Amoxicillin 350.0 1 DrMehta Physician 1202 Cetirizine 120.0 2 DrNair Dermatologist 1203 Metformin 480.0 3 DrIyer Physician 1204 Ibuprofen 90.0 4 DrPaul physician Physician	1203 Metformin 480.0 1201 Amoxicillin 350.0 1204 Ibuprofen 90.0	1201 Amoxicillin 350.0 1 DrMehta Physician 1202 Cetirizine 120.0 2 DrNair Dermatologist 1203 Metformin 480.0 3 DrIyer Physician 1204 Ibuprofen 90.0 4 DrPaul physician Cardiologist	No matching elements found.

Explanation: Filter prescriptions where the doctor's specialization matches targetSpecialization. Sort descending by cost. Records that pass the filter are printed in this order of prescriptionId: 1203, 1201, 1204.

Exam strategy: Doctor 4 is stored as 'physician' (lower case) and still has to match.

Problem A13: Complete Uncut Java Implementation

Complete file: Solution class (method + main) followed by the inner and outer POJO classes. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHOD FIRST
    public static Prescription[] findPrescriptionBySpecialization(Prescription[] arr, String targetSpecialization) {
        List<Prescription> list = new ArrayList<>();
        for (Prescription o : arr) {
            Doctor d = o.getDoctorDetails(); // inner object via aggregation
            if (d.getSpecialization().equalsIgnoreCase(targetSpecialization)) {
                list.add(o);
            }
        }
        if (list.isEmpty()) return null;
        // descending by cost
        list.sort((a, b) ->
                Double.compare(b.getCost(), a.getCost()));
        return list.toArray(new Prescription[0]);
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Prescription[] arr = new Prescription[4];
        for (int i = 0; i < 4; i++) {
            int prescriptionId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String medicineName = sc.nextLine();
            double cost = sc.nextDouble();
            sc.nextLine(); // flush newline buffer
            // ---- inner (aggregated) object attributes ----
            int doctorId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String doctorName = sc.nextLine();
            String specialization = sc.nextLine();
            Doctor doctorDetailsObj = new Doctor(doctorId, doctorName, specialization);
            arr[i] = new Prescription(prescriptionId, medicineName, cost, doctorDetailsObj);
        }
        // ---- filter parameters ----
        String targetSpecialization = sc.nextLine();

        Prescription[] result = findPrescriptionBySpecialization(arr, targetSpecialization);
        if (result != null) {
            for (Prescription o : result) {
                System.out.println(o.getPrescriptionId() + " " + o.getMedicineName() + " " + o.getCost());
            }
        } else {
            System.out.println("No matching elements found.");
        }
        sc.close();
    }
}

// 3. INNER (AGGREGATED) ENTITY POJO
class Doctor {
    private int doctorId;
    private String doctorName;
    private String specialization;

    public Doctor(int doctorId, String doctorName, String specialization) {
        this.doctorId = doctorId;
        this.doctorName = doctorName;
        this.specialization = specialization;
    }

    public int getDoctorId() { return doctorId; }
    public void setDoctorId(int doctorId) { this.doctorId = doctorId; }
    public String getDoctorName() { return doctorName; }
    public void setDoctorName(String doctorName) { this.doctorName = doctorName; }
    public String getSpecialization() { return specialization; }
    public void setSpecialization(String specialization) { this.specialization = specialization; }
}

// 4. OUTER ENTITY POJO (HAS-A inner object)
class Prescription {
    private int prescriptionId;
    private String medicineName;
    private double cost;
    private Doctor doctorDetails;

    public Prescription(int prescriptionId, String medicineName, double cost,
            Doctor doctorDetails) {
        this.prescriptionId = prescriptionId;
        this.medicineName = medicineName;
        this.cost = cost;
        this.doctorDetails = doctorDetails;
    }

    public int getPrescriptionId() { return prescriptionId; }
    public void setPrescriptionId(int prescriptionId) { this.prescriptionId = prescriptionId; }
    public String getMedicineName() { return medicineName; }
```

```
    public void setMedicineName(String medicineName) { this.medicineName = medicineName; }
    public double getCost() { return cost; }
    public void setCost(double cost) { this.cost = cost; }
    public Doctor getDoctorDetails() { return doctorDetails; }
    public void setDoctorDetails(Doctor doctorDetails) { this.doctorDetails = doctorDetails; }
}
```

Problem A14: Subscription & Account Tier - Statement and I/O

1. Entity classes (Aggregation: the outer class holds an object of the inner class)

Class	Private attributes (in this order)	Role
AccountTier	int tierId, String tierName, int monthlyMaxHours	Inner (aggregated) entity
Subscription	int subId, String userEmail, int activeDaysRemaining, **AccountTier tierDetails**	Outer entity: HAS-A AccountTier

Each class needs a parameterized constructor and a getter and setter for every attribute. The outer constructor takes the inner object as its LAST argument.

2. Static business method (class Solution)

`Subscription[] findSubscribersWithHighUsageTier(Subscription[] arr, String targetTier, int minDays)`

Filter subscriptions matching tierName AND activeDaysRemaining >= minDays. Sort ascending by subId. If nothing matches, return null.

3. Main execution rules

Read 4 Subscription objects. For each one, read the outer values (subId, userEmail, activeDaysRemaining) then the inner values (tierId, tierName, monthlyMaxHours), build the AccountTier object, then build the Subscription. After the 4 objects read: targetTier, minDays. Print subId, userEmail, activeDaysRemaining of every result object on one line (space separated), or "No matching elements found." when the result is null.

Test runs (real output of the code on the next pages) - left: sample; right: edge case with no match

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
1304 d@mail.com 40 4 Premium 500 1301 a@mail.com 25 1 Premium 500 1302 b@mail.com 10 2 Basic 100 1303 c@mail.com 5 3 Premium 500 Premium 20	1301 a@mail.com 25 1304 d@mail.com 40	1304 d@mail.com 40 4 Premium 500 1301 a@mail.com 25 1 Premium 500 1302 b@mail.com 10 2 Basic 100 1303 c@mail.com 5 3 Premium 500 Enterprise 1	No matching elements found.

Explanation: Filter subscriptions matching tierName AND activeDaysRemaining >= minDays. Sort ascending by subId. Records that pass the filter are printed in this order of subId: 1301, 1304.

Exam strategy: Email lines contain '@' and '.', so always read them with nextLine(), never next().

Problem A14: Complete Uncut Java Implementation

Complete file: Solution class (method + main) followed by the inner and outer POJO classes. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHOD FIRST
    public static Subscription[] findSubscribersWithHighUsageTier(Subscription[] arr, String targetTier, int minDays) {
        List<Subscription> list = new ArrayList<>();
        for (Subscription o : arr) {
            AccountTier d = o.getTierDetails(); // inner object via aggregation
            if (d.getTierName().equalsIgnoreCase(targetTier) && o.getActiveDaysRemaining() >= minDays) {
                list.add(o);
            }
        }
        if (list.isEmpty()) return null;
        // ascending by subId
        list.sort((a, b) ->
                Integer.compare(a.getSubId(), b.getSubId()));
        return list.toArray(new Subscription[0]);
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Subscription[] arr = new Subscription[4];
        for (int i = 0; i < 4; i++) {
            int subId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String userEmail = sc.nextLine();
            int activeDaysRemaining = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            // ---- inner (aggregated) object attributes ----
            int tierId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String tierName = sc.nextLine();
            int monthlyMaxHours = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            AccountTier tierDetailsObj = new AccountTier(tierId, tierName, monthlyMaxHours);
            arr[i] = new Subscription(subId, userEmail, activeDaysRemaining, tierDetailsObj);
        }
        // ---- filter parameters ----
        String targetTier = sc.nextLine();
        int minDays = sc.nextInt();

        Subscription[] result = findSubscribersWithHighUsageTier(arr, targetTier, minDays);
        if (result != null) {
            for (Subscription o : result) {
                System.out.println(o.getSubId() + " " + o.getUserEmail() + " " + o.getActiveDaysRemaining());
            }
        } else {
            System.out.println("No matching elements found.");
        }
        sc.close();
    }
}

// 3. INNER (AGGREGATED) ENTITY POJO
class AccountTier {
    private int tierId;
    private String tierName;
    private int monthlyMaxHours;

    public AccountTier(int tierId, String tierName, int monthlyMaxHours) {
        this.tierId = tierId;
        this.tierName = tierName;
        this.monthlyMaxHours = monthlyMaxHours;
    }

    public int getTierId() { return tierId; }
    public void setTierId(int tierId) { this.tierId = tierId; }
    public String getTierName() { return tierName; }
    public void setTierName(String tierName) { this.tierName = tierName; }
    public int getMonthlyMaxHours() { return monthlyMaxHours; }
    public void setMonthlyMaxHours(int monthlyMaxHours) { this.monthlyMaxHours = monthlyMaxHours; }
}

// 4. OUTER ENTITY POJO (HAS-A inner object)
class Subscription {
    private int subId;
    private String userEmail;
    private int activeDaysRemaining;
    private AccountTier tierDetails;

    public Subscription(int subId, String userEmail, int activeDaysRemaining,
            AccountTier tierDetails) {
        this.subId = subId;
        this.userEmail = userEmail;
        this.activeDaysRemaining = activeDaysRemaining;
        this.tierDetails = tierDetails;
    }

    public int getSubId() { return subId; }
```

```
    public void setSubId(int subId) { this.subId = subId; }
    public String getUserEmail() { return userEmail; }
    public void setUserEmail(String userEmail) { this.userEmail = userEmail; }
    public int getActiveDaysRemaining() { return activeDaysRemaining; }
    public void setActiveDaysRemaining(int activeDaysRemaining) { this.activeDaysRemaining = activeDaysRemaining; }
    public AccountTier getTierDetails() { return tierDetails; }
    public void setTierDetails(AccountTier tierDetails) { this.tierDetails = tierDetails; }
}
```

Problem A15: Inventory Item & Warehouse Location - Statement and I/O

1. Entity classes (Aggregation: the outer class holds an object of the inner class)

Class	Private attributes (in this order)	Role
Warehouse	int warehouseId, String zoneName, int safetyRating	Inner (aggregated) entity
InventoryItem	int itemId, String itemName, double unitPrice, **Warehouse warehouseDetails**	Outer entity: HAS-A Warehouse

Each class needs a parameterized constructor and a getter and setter for every attribute. The outer constructor takes the inner object as its LAST argument.

2. Static business method (class Solution)

`InventoryItem[] findItemsInSafeZones(InventoryItem[] arr, String targetZone, int minSafetyRating)`

Filter inventory items matching zoneName AND safetyRating >= minSafetyRating. Sort descending by unitPrice. If nothing matches, return null.

3. Main execution rules

Read 4 InventoryItem objects. For each one, read the outer values (itemId, itemName, unitPrice) then the inner values (warehouseId, zoneName, safetyRating), build the Warehouse object, then build the InventoryItem. After the 4 objects read: targetZone, minSafetyRating. Print itemId, itemName, unitPrice of every result object on one line (space separated), or "No matching elements found." when the result is null.

Test runs (real output of the code on the next pages) - left: sample; right: edge case with no match

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
1401 Laptop 55000.0 1 ZoneA 9 1402 Printer 12000.0 2 ZoneB 6 1403 Router 3500.0 3 ZoneA 8 1404 Server 150000.0 4 zonea 9 ZoneA 8	1404 Server 150000.0 1401 Laptop 55000.0 1403 Router 3500.0	1401 Laptop 55000.0 1 ZoneA 9 1402 Printer 12000.0 2 ZoneB 6 1403 Router 3500.0 3 ZoneA 8 1404 Server 150000.0 4 zonea 9 ZoneC 5	No matching elements found.

Explanation: Filter inventory items matching zoneName AND safetyRating >= minSafetyRating. Sort descending by unitPrice. Records that pass the filter are printed in this order of itemId: 1404, 1401, 1403.

Exam strategy: Item 1404 is stored as 'zonea': a == or equals() comparison would silently miss it.

Problem A15: Complete Uncut Java Implementation

Complete file: Solution class (method + main) followed by the inner and outer POJO classes. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHOD FIRST
    public static InventoryItem[] findItemsInSafeZones(InventoryItem[] arr, String targetZone, int minSafetyRating) {
        List<InventoryItem> list = new ArrayList<>();
        for (InventoryItem o : arr) {
            Warehouse d = o.getWarehouseDetails(); // inner object via aggregation
            if (d.getZoneName().equalsIgnoreCase(targetZone) && d.getSafetyRating() >= minSafetyRating) {
                list.add(o);
            }
        }
        if (list.isEmpty()) return null;
        // descending by unitPrice
        list.sort((a, b) ->
                Double.compare(b.getUnitPrice(), a.getUnitPrice()));
        return list.toArray(new InventoryItem[0]);
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        InventoryItem[] arr = new InventoryItem[4];
        for (int i = 0; i < 4; i++) {
            int itemId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String itemName = sc.nextLine();
            double unitPrice = sc.nextDouble();
            sc.nextLine(); // flush newline buffer
            // ---- inner (aggregated) object attributes ----
            int warehouseId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String zoneName = sc.nextLine();
            int safetyRating = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            Warehouse warehouseDetailsObj = new Warehouse(warehouseId, zoneName, safetyRating);
            arr[i] = new InventoryItem(itemId, itemName, unitPrice, warehouseDetailsObj);
        }
        // ---- filter parameters ----
        String targetZone = sc.nextLine();
        int minSafetyRating = sc.nextInt();

        InventoryItem[] result = findItemsInSafeZones(arr, targetZone, minSafetyRating);
        if (result != null) {
            for (InventoryItem o : result) {
                System.out.println(o.getItemId() + " " + o.getItemName() + " " + o.getUnitPrice());
            }
        } else {
            System.out.println("No matching elements found.");
        }
        sc.close();
    }
}

// 3. INNER (AGGREGATED) ENTITY POJO
class Warehouse {
    private int warehouseId;
    private String zoneName;
    private int safetyRating;

    public Warehouse(int warehouseId, String zoneName, int safetyRating) {
        this.warehouseId = warehouseId;
        this.zoneName = zoneName;
        this.safetyRating = safetyRating;
    }

    public int getWarehouseId() { return warehouseId; }
    public void setWarehouseId(int warehouseId) { this.warehouseId = warehouseId; }
    public String getZoneName() { return zoneName; }
    public void setZoneName(String zoneName) { this.zoneName = zoneName; }
    public int getSafetyRating() { return safetyRating; }
    public void setSafetyRating(int safetyRating) { this.safetyRating = safetyRating; }
}

// 4. OUTER ENTITY POJO (HAS-A inner object)
class InventoryItem {
    private int itemId;
    private String itemName;
    private double unitPrice;
    private Warehouse warehouseDetails;

    public InventoryItem(int itemId, String itemName, double unitPrice,
            Warehouse warehouseDetails) {
        this.itemId = itemId;
        this.itemName = itemName;
        this.unitPrice = unitPrice;
        this.warehouseDetails = warehouseDetails;
    }

    public int getItemId() { return itemId; }
```

```
    public void setItemId(int itemId) { this.itemId = itemId; }
    public String getItemName() { return itemName; }
    public void setItemName(String itemName) { this.itemName = itemName; }
    public double getUnitPrice() { return unitPrice; }
    public void setUnitPrice(double unitPrice) { this.unitPrice = unitPrice; }
    public Warehouse getWarehouseDetails() { return warehouseDetails; }
    public void setWarehouseDetails(Warehouse warehouseDetails) { this.warehouseDetails = warehouseDetails; }
}
```

Problem A16: Bank Account & Branch (single MAX object) - Statement and I/O

1. Entity classes (Aggregation: the outer class holds an object of the inner class)

Class	Private attributes (in this order)	Role
Branch	int branchId, String branchName, String city	Inner (aggregated) entity
Account	int accountNo, String holderName, double balance, **Branch branchDetails**	Outer entity: HAS-A Branch

Each class needs a parameterized constructor and a getter and setter for every attribute. The outer constructor takes the inner object as its LAST argument.

2. Static business method (class Solution)

`Account findAccountWithMaxBalanceByCity(Account[] arr, String targetCity)`

Return the single Account with the MAXIMUM balance among accounts whose branch city matches targetCity (case-insensitive). Return null if none.

3. Main execution rules

Read 4 Account objects. For each one, read the outer values (accountNo, holderName, balance) then the inner values (branchId, branchName, city), build the Branch object, then build the Account. After the 4 objects read: targetCity. Print the object's accountNo, holderName, balance on one line separated by spaces, or "No account found".

Test runs (real output of the code on the next pages) - left: sample; right: edge case with no match

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
5001 Mohan 42000.5 1 MainBranch Kolkata 5002 Nisha 91000.0 2 CityCentre Mumbai 5003 Om 67000.0 3 SaltLake Kolkata 5004 Pooja 15000.0 4 Dadar Mumbai Kolkata	5003 Om 67000.0	5001 Mohan 42000.5 1 MainBranch Kolkata 5002 Nisha 91000.0 2 CityCentre Mumbai 5003 Om 67000.0 3 SaltLake Kolkata 5004 Pooja 15000.0 4 Dadar Mumbai Pune	No account found

Explanation: Return the single Account with the MAXIMUM balance among accounts whose branch city matches targetCity (case-insensitive). Return null if none. The winner for the sample input is 5003 Om 67000.0.

Exam strategy: Return type is ONE object, not an array: use the 'best = null' skeleton and print only if best != null.

Problem A16: Complete Uncut Java Implementation

Complete file: Solution class (method + main) followed by the inner and outer POJO classes. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHOD FIRST
    public static Account findAccountWithMaxBalanceByCity(Account[] arr, String targetCity) {
        Account best = null;
        for (Account o : arr) {
            Branch d = o.getBranchDetails(); // inner object via aggregation
            if (d.getCity().equalsIgnoreCase(targetCity)) {
                if (best == null || o.getBalance() > best.getBalance()) {
                    best = o;
                }
            }
        }
        return best;
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Account[] arr = new Account[4];
        for (int i = 0; i < 4; i++) {
            int accountNo = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String holderName = sc.nextLine();
            double balance = sc.nextDouble();
            sc.nextLine(); // flush newline buffer
            // ---- inner (aggregated) object attributes ----
            int branchId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String branchName = sc.nextLine();
            String city = sc.nextLine();
            Branch branchDetailsObj = new Branch(branchId, branchName, city);
            arr[i] = new Account(accountNo, holderName, balance, branchDetailsObj);
        }
        // ---- filter parameters ----
        String targetCity = sc.nextLine();

        Account res = findAccountWithMaxBalanceByCity(arr, targetCity);
        if (res != null) {
            System.out.println(res.getAccountNo() + " " + res.getHolderName() + " " + res.getBalance());
        } else {
            System.out.println("No account found");
        }
        sc.close();
    }
}

// 3. INNER (AGGREGATED) ENTITY POJO
class Branch {
    private int branchId;
    private String branchName;
    private String city;

    public Branch(int branchId, String branchName, String city) {
        this.branchId = branchId;
        this.branchName = branchName;
        this.city = city;
    }

    public int getBranchId() { return branchId; }
    public void setBranchId(int branchId) { this.branchId = branchId; }
    public String getBranchName() { return branchName; }
    public void setBranchName(String branchName) { this.branchName = branchName; }
    public String getCity() { return city; }
    public void setCity(String city) { this.city = city; }
}

// 4. OUTER ENTITY POJO (HAS-A inner object)
class Account {
    private int accountNo;
    private String holderName;
    private double balance;
    private Branch branchDetails;

    public Account(int accountNo, String holderName, double balance,
            Branch branchDetails) {
        this.accountNo = accountNo;
        this.holderName = holderName;
        this.balance = balance;
        this.branchDetails = branchDetails;
    }

    public int getAccountNo() { return accountNo; }
    public void setAccountNo(int accountNo) { this.accountNo = accountNo; }
    public String getHolderName() { return holderName; }
    public void setHolderName(String holderName) { this.holderName = holderName; }
    public double getBalance() { return balance; }
    public void setBalance(double balance) { this.balance = balance; }
    public Branch getBranchDetails() { return branchDetails; }
```

```
    public void setBranchDetails(Branch branchDetails) { this.branchDetails = branchDetails; }
}
```

Problem A17: Customer & Address (COUNT result) - Statement and I/O

1. Entity classes (Aggregation: the outer class holds an object of the inner class)

Class	Private attributes (in this order)	Role
Address	int addressId, String city, int pincode	Inner (aggregated) entity
Customer	int customerId, String customerName, int creditScore, **Address addressDetails**	Outer entity: HAS-A Address

Each class needs a parameterized constructor and a getter and setter for every attribute. The outer constructor takes the inner object as its LAST argument.

2. Static business method (class Solution)

```
int countCustomersByCity(Customer[] arr, String targetCity, int minScore)
```

Return the COUNT of customers whose address city matches targetCity (case-insensitive) AND creditScore >= minScore. Return 0 if none.

3. Main execution rules

Read 4 Customer objects. For each one, read the outer values (customerId, customerName, creditScore) then the inner values (addressId, city, pincode), build the Address object, then build the Customer. After the 4 objects read: targetCity, minScore. Print the count, or "No matching customers found" when it is 0.

Test runs (real output of the code on the next pages) - left: sample; right: edge case with no match

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
1 Anil 720 11 Pune 411001 2 Bina 640 12 Pune 411002 3 Chetan 780 13 Delhi 110001 4 Disha 700 14 pune 411004 Pune 700	2	1 Anil 720 11 Pune 411001 2 Bina 640 12 Pune 411002 3 Chetan 780 13 Delhi 110001 4 Disha 700 14 pune 411004 Pune 900	No matching customers found

Explanation: Return the COUNT of customers whose address city matches targetCity (case-insensitive) AND creditScore >= minScore. Return 0 if none. The count for the sample input is 2.

Exam strategy: Return 0 (not null) for a count, and print the 'not found' message in main when the count is 0.

Problem A17: Complete Uncut Java Implementation

Complete file: Solution class (method + main) followed by the inner and outer POJO classes. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHOD FIRST
    public static int countCustomersByCity(Customer[] arr, String targetCity, int minScore) {
        int count = 0;
        for (Customer o : arr) {
            Address d = o.getAddressDetails(); // inner object via aggregation
            if (d.getCity().equalsIgnoreCase(targetCity) && o.getCreditScore() >= minScore) {
                count++;
            }
        }
        return count;
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Customer[] arr = new Customer[4];
        for (int i = 0; i < 4; i++) {
            int customerId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String customerName = sc.nextLine();
            int creditScore = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            // ---- inner (aggregated) object attributes ----
            int addressId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String city = sc.nextLine();
            int pincode = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            Address addressDetailsObj = new Address(addressId, city, pincode);
            arr[i] = new Customer(customerId, customerName, creditScore, addressDetailsObj);
        }
        // ---- filter parameters ----
        String targetCity = sc.nextLine();
        int minScore = sc.nextInt();

        int count = countCustomersByCity(arr, targetCity, minScore);
        if (count > 0) {
            System.out.println(count);
        } else {
            System.out.println("No matching customers found");
        }
        sc.close();
    }
}

// 3. INNER (AGGREGATED) ENTITY POJO
class Address {
    private int addressId;
    private String city;
    private int pincode;

    public Address(int addressId, String city, int pincode) {
        this.addressId = addressId;
        this.city = city;
        this.pincode = pincode;
    }

    public int getAddressId() { return addressId; }
    public void setAddressId(int addressId) { this.addressId = addressId; }
    public String getCity() { return city; }
    public void setCity(String city) { this.city = city; }
    public int getPincode() { return pincode; }
    public void setPincode(int pincode) { this.pincode = pincode; }
}

// 4. OUTER ENTITY POJO (HAS-A inner object)
class Customer {
    private int customerId;
    private String customerName;
    private int creditScore;
    private Address addressDetails;

    public Customer(int customerId, String customerName, int creditScore,
            Address addressDetails) {
        this.customerId = customerId;
        this.customerName = customerName;
        this.creditScore = creditScore;
        this.addressDetails = addressDetails;
    }

    public int getCustomerId() { return customerId; }
    public void setCustomerId(int customerId) { this.customerId = customerId; }
    public String getCustomerName() { return customerName; }
    public void setCustomerName(String customerName) { this.customerName = customerName; }
    public int getCreditScore() { return creditScore; }
    public void setCreditScore(int creditScore) { this.creditScore = creditScore; }
    public Address getAddressDetails() { return addressDetails; }
```

```
    public void setAddressDetails(Address addressDetails) { this.addressDetails = addressDetails; }
}
```

Problem A18: Team & Coach Performance - Statement and I/O

1. Entity classes (Aggregation: the outer class holds an object of the inner class)

Class	Private attributes (in this order)	Role
Coach	int coachId, String coachName, double winPercentage	Inner (aggregated) entity
Team	int teamId, String teamName, int points, **Coach coachDetails**	Outer entity: HAS-A Coach

Each class needs a parameterized constructor and a getter and setter for every attribute. The outer constructor takes the inner object as its LAST argument.

2. Static business method (class Solution)

`Team[] findTeamsByCoachWinRate(Team[] arr, double minWinPercentage)`

Filter teams whose coach has winPercentage >= minWinPercentage. Sort descending by team points. If nothing matches, return null.

3. Main execution rules

Read 4 Team objects. For each one, read the outer values (teamId, teamName, points) then the inner values (coachId, coachName, winPercentage), build the Coach object, then build the Team. After the 4 objects read: minWinPercentage. Print teamId, teamName, points of every result object on one line (space separated), or "No matching teams found" when the result is null.

Test runs (real output of the code on the next pages) - left: sample; right: edge case with no match

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
21 Tigers 58 1 Rao 72.5 22 Lions 64 2 Sen 55.0 23 Eagles 71 3 Das 80.0 24 Wolves 49 4 Roy 65.0 60.0	23 Eagles 71 21 Tigers 58 24 Wolves 49	21 Tigers 58 1 Rao 72.5 22 Lions 64 2 Sen 55.0 23 Eagles 71 3 Das 80.0 24 Wolves 49 4 Roy 65.0 95.0	No matching teams found

Explanation: Filter teams whose coach has winPercentage >= minWinPercentage. Sort descending by team points. Records that pass the filter are printed in this order of teamId: 23, 21, 24.

Exam strategy: Same skeleton as Problems A1 to A15; only the field names change. Practise typing it from memory.

Problem A18: Complete Uncut Java Implementation

Complete file: Solution class (method + main) followed by the inner and outer POJO classes. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHOD FIRST
    public static Team[] findTeamsByCoachWinRate(Team[] arr, double minWinPercentage) {
        List<Team> list = new ArrayList<>();
        for (Team o : arr) {
            Coach d = o.getCoachDetails(); // inner object via aggregation
            if (d.getWinPercentage() >= minWinPercentage) {
                list.add(o);
            }
        }
        if (list.isEmpty()) return null;
        // descending by points
        list.sort((a, b) ->
                Integer.compare(b.getPoints(), a.getPoints()));
        return list.toArray(new Team[0]);
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Team[] arr = new Team[4];
        for (int i = 0; i < 4; i++) {
            int teamId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String teamName = sc.nextLine();
            int points = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            // ---- inner (aggregated) object attributes ----
            int coachId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String coachName = sc.nextLine();
            double winPercentage = sc.nextDouble();
            sc.nextLine(); // flush newline buffer
            Coach coachDetailsObj = new Coach(coachId, coachName, winPercentage);
            arr[i] = new Team(teamId, teamName, points, coachDetailsObj);
        }
        // ---- filter parameters ----
        double minWinPercentage = sc.nextDouble();

        Team[] result = findTeamsByCoachWinRate(arr, minWinPercentage);
        if (result != null) {
            for (Team o : result) {
                System.out.println(o.getTeamId() + " " + o.getTeamName() + " " + o.getPoints());
            }
        } else {
            System.out.println("No matching teams found");
        }
        sc.close();
    }
}

// 3. INNER (AGGREGATED) ENTITY POJO
class Coach {
    private int coachId;
    private String coachName;
    private double winPercentage;

    public Coach(int coachId, String coachName, double winPercentage) {
        this.coachId = coachId;
        this.coachName = coachName;
        this.winPercentage = winPercentage;
    }

    public int getCoachId() { return coachId; }
    public void setCoachId(int coachId) { this.coachId = coachId; }
    public String getCoachName() { return coachName; }
    public void setCoachName(String coachName) { this.coachName = coachName; }
    public double getWinPercentage() { return winPercentage; }
    public void setWinPercentage(double winPercentage) { this.winPercentage = winPercentage; }
}

// 4. OUTER ENTITY POJO (HAS-A inner object)
class Team {
    private int teamId;
    private String teamName;
    private int points;
    private Coach coachDetails;

    public Team(int teamId, String teamName, int points, Coach coachDetails) {
        this.teamId = teamId;
        this.teamName = teamName;
        this.points = points;
        this.coachDetails = coachDetails;
    }

    public int getTeamId() { return teamId; }
    public void setTeamId(int teamId) { this.teamId = teamId; }
    public String getTeamName() { return teamName; }
```

```
    public void setTeamName(String teamName) { this.teamName = teamName; }
    public int getPoints() { return points; }
    public void setPoints(int points) { this.points = points; }
    public Coach getCoachDetails() { return coachDetails; }
    public void setCoachDetails(Coach coachDetails) { this.coachDetails = coachDetails; }
}
```

Problem A19: Tablet & Display (MAX by inner attribute) - Statement and I/O

1. Entity classes (Aggregation: the outer class holds an object of the inner class)

Class	Private attributes (in this order)	Role
Display	int displayId, String panelType, double sizeInches	Inner (aggregated) entity
Tablet	int tabletId, String model, double price, **Display displayDetails**	Outer entity: HAS-A Display

Each class needs a parameterized constructor and a getter and setter for every attribute. The outer constructor takes the inner object as its LAST argument.

2. Static business method (class Solution)

`Tablet findTabletWithLargestDisplay(Tablet[] arr, String targetPanel)`

Return the Tablet whose Display has the LARGEST sizeInches among tablets with panelType equal to targetPanel (case-insensitive). Return null if none.

3. Main execution rules

Read 4 Tablet objects. For each one, read the outer values (tabletId, model, price) then the inner values (displayId, panelType, sizeInches), build the Display object, then build the Tablet. After the 4 objects read: targetPanel. Print the object's tabletId, model, price on one line separated by spaces, or "No tablet found".

Test runs (real output of the code on the next pages) - left: sample; right: edge case with no match

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
31 TabX 24999.0 1 OLED 10.4 32 TabPro 54999.0 2 OLED 12.9 33 TabLite 12999.0 3 LCD 8.7 34 TabMax 39999.0 4 OLED 11.0 OLED	32 TabPro 54999.0	31 TabX 24999.0 1 OLED 10.4 32 TabPro 54999.0 2 OLED 12.9 33 TabLite 12999.0 3 LCD 8.7 34 TabMax 39999.0 4 OLED 11.0 AMOLED	No tablet found

Explanation: Return the Tablet whose Display has the LARGEST sizeInches among tablets with panelType equal to targetPanel (case-insensitive). Return null if none. The winner for the sample input is 32 TabPro 54999.0.

Exam strategy: The max is compared on the INNER value: best.getDisplayDetails().getSizeInches().

Problem A19: Complete Uncut Java Implementation

Complete file: Solution class (method + main) followed by the inner and outer POJO classes. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHOD FIRST
    public static Tablet findTabletWithLargestDisplay(Tablet[] arr, String targetPanel) {
        Tablet best = null;
        for (Tablet o : arr) {
            Display d = o.getDisplayDetails(); // inner object via aggregation
            if (d.getPanelType().equalsIgnoreCase(targetPanel)) {
                if (best == null || o.getDisplayDetails().getSizeInches() > best.getDisplayDetails().getSizeInches()) {
                    best = o;
                }
            }
        }
        return best;
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Tablet[] arr = new Tablet[4];
        for (int i = 0; i < 4; i++) {
            int tabletId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String model = sc.nextLine();
            double price = sc.nextDouble();
            sc.nextLine(); // flush newline buffer
            // ---- inner (aggregated) object attributes ----
            int displayId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String panelType = sc.nextLine();
            double sizeInches = sc.nextDouble();
            sc.nextLine(); // flush newline buffer
            Display displayDetailsObj = new Display(displayId, panelType, sizeInches);
            arr[i] = new Tablet(tabletId, model, price, displayDetailsObj);
        }
        // ---- filter parameters ----
        String targetPanel = sc.nextLine();

        Tablet res = findTabletWithLargestDisplay(arr, targetPanel);
        if (res != null) {
            System.out.println(res.getTabletId() + " " + res.getModel() + " " + res.getPrice());
        } else {
            System.out.println("No tablet found");
        }
        sc.close();
    }
}

// 3. INNER (AGGREGATED) ENTITY POJO
class Display {
    private int displayId;
    private String panelType;
    private double sizeInches;

    public Display(int displayId, String panelType, double sizeInches) {
        this.displayId = displayId;
        this.panelType = panelType;
        this.sizeInches = sizeInches;
    }

    public int getDisplayId() { return displayId; }
    public void setDisplayId(int displayId) { this.displayId = displayId; }
    public String getPanelType() { return panelType; }
    public void setPanelType(String panelType) { this.panelType = panelType; }
    public double getSizeInches() { return sizeInches; }
    public void setSizeInches(double sizeInches) { this.sizeInches = sizeInches; }
}

// 4. OUTER ENTITY POJO (HAS-A inner object)
class Tablet {
    private int tabletId;
    private String model;
    private double price;
    private Display displayDetails;

    public Tablet(int tabletId, String model, double price,
            Display displayDetails) {
        this.tabletId = tabletId;
        this.model = model;
        this.price = price;
        this.displayDetails = displayDetails;
    }

    public int getTabletId() { return tabletId; }
    public void setTabletId(int tabletId) { this.tabletId = tabletId; }
    public String getModel() { return model; }
    public void setModel(String model) { this.model = model; }
    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }
```

```
    public Display getDisplayDetails() { return displayDetails; }
    public void setDisplayDetails(Display displayDetails) { this.displayDetails = displayDetails; }
}
```

Problem A20: Library Member & Membership Plan - Statement and I/O

1. Entity classes (Aggregation: the outer class holds an object of the inner class)

Class	Private attributes (in this order)	Role
Membership	int planId, String planName, int validityMonths	Inner (aggregated) entity
Member	int memberId, String memberName, int booksIssued, **Membership membershipDetails**	Outer entity: HAS-A Membership

Each class needs a parameterized constructor and a getter and setter for every attribute. The outer constructor takes the inner object as its LAST argument.

2. Static business method (class Solution)

`Member[] getMembersByPlan(Member[] arr, String targetPlan, int minBooks)`

Filter members whose membership planName matches targetPlan AND booksIssued >= minBooks. Sort descending by booksIssued. If nothing matches, return null.

3. Main execution rules

Read 4 Member objects. For each one, read the outer values (memberId, memberName, booksIssued) then the inner values (planId, planName, validityMonths), build the Membership object, then build the Member. After the 4 objects read: targetPlan, minBooks. Print memberId, memberName, booksIssued of every result object on one line (space separated), or "No matching members found" when the result is null.

Test runs (real output of the code on the next pages) - left: sample; right: edge case with no match

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
41 Kavya 14 1 Gold 12 42 Lalit 6 2 Silver 6 43 Meera 22 3 Gold 12 44 Nikhil 9 4 Gold 12 Gold 10	43 Meera 22 41 Kavya 14	41 Kavya 14 1 Gold 12 42 Lalit 6 2 Silver 6 43 Meera 22 3 Gold 12 44 Nikhil 9 4 Gold 12 Platinum 1	No matching members found

Explanation: Filter members whose membership planName matches targetPlan AND booksIssued >= minBooks. Sort descending by booksIssued. Records that pass the filter are printed in this order of memberId: 43, 41.

Exam strategy: Two different objects are tested in one condition. Always read the rule twice to see which object owns each attribute.

Problem A20: Complete Uncut Java Implementation

Complete file: Solution class (method + main) followed by the inner and outer POJO classes. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHOD FIRST
    public static Member[] getMembersByPlan(Member[] arr, String targetPlan, int minBooks) {
        List<Member> list = new ArrayList<>();
        for (Member o : arr) {
            Membership d = o.getMembershipDetails(); // inner object via aggregation
            if (d.getPlanName().equalsIgnoreCase(targetPlan) && o.getBooksIssued() >= minBooks) {
                list.add(o);
            }
        }
        if (list.isEmpty()) return null;
        // descending by booksIssued
        list.sort((a, b) ->
                Integer.compare(b.getBooksIssued(), a.getBooksIssued()));
        return list.toArray(new Member[0]);
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Member[] arr = new Member[4];
        for (int i = 0; i < 4; i++) {
            int memberId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String memberName = sc.nextLine();
            int booksIssued = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            // ---- inner (aggregated) object attributes ----
            int planId = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            String planName = sc.nextLine();
            int validityMonths = sc.nextInt();
            sc.nextLine(); // flush newline buffer
            Membership membershipDetailsObj = new Membership(planId, planName, validityMonths);
            arr[i] = new Member(memberId, memberName, booksIssued, membershipDetailsObj);
        }
        // ---- filter parameters ----
        String targetPlan = sc.nextLine();
        int minBooks = sc.nextInt();

        Member[] result = getMembersByPlan(arr, targetPlan, minBooks);
        if (result != null) {
            for (Member o : result) {
                System.out.println(o.getMemberId() + " " + o.getMemberName() + " " + o.getBooksIssued());
            }
        } else {
            System.out.println("No matching members found");
        }
        sc.close();
    }
}

// 3. INNER (AGGREGATED) ENTITY POJO
class Membership {
    private int planId;
    private String planName;
    private int validityMonths;

    public Membership(int planId, String planName, int validityMonths) {
        this.planId = planId;
        this.planName = planName;
        this.validityMonths = validityMonths;
    }

    public int getPlanId() { return planId; }
    public void setPlanId(int planId) { this.planId = planId; }
    public String getPlanName() { return planName; }
    public void setPlanName(String planName) { this.planName = planName; }
    public int getValidityMonths() { return validityMonths; }
    public void setValidityMonths(int validityMonths) { this.validityMonths = validityMonths; }
}

// 4. OUTER ENTITY POJO (HAS-A inner object)
class Member {
    private int memberId;
    private String memberName;
    private int booksIssued;
    private Membership membershipDetails;

    public Member(int memberId, String memberName, int booksIssued,
            Membership membershipDetails) {
        this.memberId = memberId;
        this.memberName = memberName;
        this.booksIssued = booksIssued;
        this.membershipDetails = membershipDetails;
    }

    public int getMemberId() { return memberId; }
```

```
    public void setMemberId(int memberId) { this.memberId = memberId; }
    public String getMemberName() { return memberName; }
    public void setMemberName(String memberName) { this.memberName = memberName; }
    public int getBooksIssued() { return booksIssued; }
    public void setBooksIssued(int booksIssued) { this.booksIssued = booksIssued; }
    public Membership getMembershipDetails() { return membershipDetails; }
    public void setMembershipDetails(Membership membershipDetails) { this.membershipDetails = membershipDetails; }
}
```

Final Revision Checklist (all 35 problems)

- Template order: imports, Solution class (static methods, then main), then POJO classes.
- Constructor parameter order must match the question's attribute order exactly.
- Case-insensitive String comparison everywhere: equalsIgnoreCase().
- Return null / 0 when nothing matches, and print the exact message from the question.
- Second-highest: sort descending, return the first value strictly below the maximum.
- Aggregation: build the inner object first, pass it as the LAST outer constructor argument.
- Run the sample input and one no-match input before submitting.
- Your exam wording always wins over any handbook: copy the rule text into a comment and follow it literally.